Discrete Optimization

Exact SAT Solving for the Two-Dimensional Bandwidth Minimization Problem

--Manuscript Draft--

Manuscript Number:	DISOPT-D-26-00172
Article Type:	Full length article
Keywords:	The two-dimensional bandwidth minimization problem (2DBMP) seeks an injective embedding of a guest graph into a square grid that minimizes the maximum Manhattan distance over its edges. Heuristic methods can provide strong upper bounds, but these bounds do
Corresponding Author:	Khanh To Van VNU University of Engineering and Technology Faculty of Information Technology
First Author:	Minh Pham Quang
Order of Authors:	Minh Pham Quang
	Nghia Dao Xuan
	Khanh To Van
Abstract:	

Powered by Editorial Manager® and ProduXion Manager® from Aries Systems Corporation

Manuscript Click here to access/download;Manuscript;manuscript.pdf

Click here to view linked References

Exact SAT Solving for the Two-Dimensional Bandwidth Minimization Problem

Pham Quang Minh, Dao Xuan Nghia and To Van Khanh*

Faculty of Information Technology, VNU University of Engineering and Technology, 144 Xuan Thuy, Cau Giay Ward, Hanoi, 100000, Vietnam

ARTICLE INFO

Keywords:
2D bandwidth minimization
graph layout
SAT solving
incremental SAT
exact optimization

ABSTRACT

The two-dimensional bandwidth minimization problem (2DBMP) seeks an injective embedding of a guest graph into a square grid that minimizes the maximum Manhattan distance over its edges. Heuristic methods can provide strong upper bounds, but these bounds do not by themselves certify optimality. We present an efficient exact SAT-based approach for 2DBMP that incrementally searches for the minimum feasible bandwidth and certifies optimality through satisfiability and unsatisfiability results. On the standard $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid, under a 3600 s time limit, the proposed SAT approach certifies optimal bandwidths for 41 of 43 Regular instances and 42 of 93 Harwell–Boeing instances, achieving substantially broader optimality certification within a one-hour time limit than a previous exact approach evaluated with a 72-hour time limit. In addition, it certifies three bandwidth values that improve all previously published comparison values considered in this study and establishes all three as optimal. We further evaluate the approach on alternative host geometries, namely $2\times\lceil n/2\rceil$ and $n\times n$ grids, to assess its effectiveness beyond the standard host. Overall, the results demonstrate that the proposed SAT approach provides an effective exact method for the small- and medium-sized benchmark instances considered in this study, with fewer than 400 vertices, while heuristic methods remain important for larger and more challenging instances.

1. Introduction

Graph layout is an important family of combinatorial optimization problems concerned with arranging the vertices of a graph on a target structure under different optimization objectives. This family of problems has broad practical relevance, with representative applications in VLSI design, telecommunication architectures, parallel processing systems, sparse matrix ordering, and matrix decomposition [5, 15, 23]. Within this family, an important line of research focuses on minimizing the maximum host-graph distance induced between adjacent vertices. The two-dimensional bandwidth minimization problem (2DBMP) is a representative problem in this line because a graph is arranged on a two-dimensional grid and edge lengths are measured by the Manhattan distance.

The problem considered in this paper is 2DBMP on the standard square grid of side $h = \lceil\sqrt{n}\rceil$ [3, 23]. Given an input graph $G = (V, E)$ with n vertices, each vertex must be assigned to a distinct grid position. The objective is to minimize the maximum Manhattan distance between adjacent vertices after the graph is placed on the grid. In the classical one-dimensional bandwidth minimization problem, each vertex receives one label and the distance between adjacent vertices is the absolute difference between labels. In 2DBMP, each vertex is identified by two grid coordinates. Consequently, moving from a path host to a two-dimensional grid changes both the geometry of feasible embeddings and the structure of the distance constraints.

The literature on 2DBMP has progressed along complementary theoretical, exact, and heuristic lines [3, 15, 23, 26]. Theoretical studies established lower and upper bounds and exact values for selected graph classes under broader host-grid formulations [15–17]. On the heuristic side, recent methods have continued to improve the upper bounds reported for benchmark instances [25, 26]. These methods terminate according to algorithm-specific stopping criteria, such as time or iteration limits, and return the best solution found [3, 25, 26]. Because these heuristic searches do not provide certificates that exclude better embeddings, an improved best-known value is not necessarily a certified optimal value. Exact methods complement heuristic search by providing optimality certificates, but they generally require substantially greater computational effort in time and memory [23]. Both exact and heuristic approaches have been studied for the Manhattan-distance problem [23] and the related Chebyshev-distance variant [4]. Prior studies have

*Corresponding author
24020239@vnu.edu.vn (P.Q. Minh); 25025081@vnu.edu.vn (D.X. Nghia); khanhtv@vnu.edu.vn (T.V. Khanh)
ORCID(s): 0009-0008-3591-3076 (P.Q. Minh); 0009-0001-9628-636X (D.X. Nghia); 0009-0008-1907-7848 (T.V. Khanh)

also considered alternative host-grid dimensions, including $2 \times \lceil n/2 \rceil$ rectangular grids [13] and $n \times n$ grids [15, 16]. These formulations broaden the range of host-grid settings and motivate further study of alternative host geometries. On the widely used Regular and Harwell–Boeing benchmarks [3, 23, 26], the exact results reported by Rodríguez-García et al. [23] certified most Regular instances but left optimality unresolved for most Harwell–Boeing instances. Consequently, tightening bounds on unresolved instances and certifying additional optimal values remain important goals for 2DBMP.

Modern SAT solving has become an effective approach for a wide range of practical problems [2, 8]. Its applications include exact methods for graph-related problems such as graph coloring, antibandwidth, and bandwidth coloring [6, 7, 20]. This makes SAT a natural framework for 2DBMP because vertex placements, occupancy requirements, and bandwidth limits can be represented by Boolean variables and clauses. Testing the resulting formula for successive values of K converts the optimization problem into a sequence of related SAT decision problems.

This paper addresses the certification gap with a SAT-based exact approach for standard 2DBMP. The proposed SAT encoding combines components for position assignment, sequential occupancy, cutoff-aware distance, symmetry breaking through an anchor vertex, and bandwidth enforcement. During the descending threshold search, the incremental procedure retains the fixed clauses and solver state while adding the bandwidth component Δ_K for each tested value of K. Experiments on the small and medium instances considered in this study, comprising 43 Regular and 93 Harwell–Boeing instances, show that the proposed method achieves competitive exact certification relative to the locally evaluated commercial solvers and extends the exact results available in the literature. Under the 3600 s time limit on the standard $\lceil \sqrt{n} \rceil \times \lceil \sqrt{n} \rceil$ host grid, it establishes three new certified optimal values for Harwell–Boeing instances. Each value is lower than every available published comparison bandwidth considered in this study and every feasible incumbent returned by Gurobi, CPLEX MIP, and CPLEX CP for the corresponding instance.

The remainder of the paper is organized as follows. Section 2 defines 2DBMP, presents an integer linear programming (ILP) formulation, and reviews the directly relevant literature. Section 3 presents the SAT encoding and the incremental solving scheme. Section 4 reports the computational experiments. Section 5 concludes the paper.

2. Background

2.1. Problem Definition and ILP Formulation

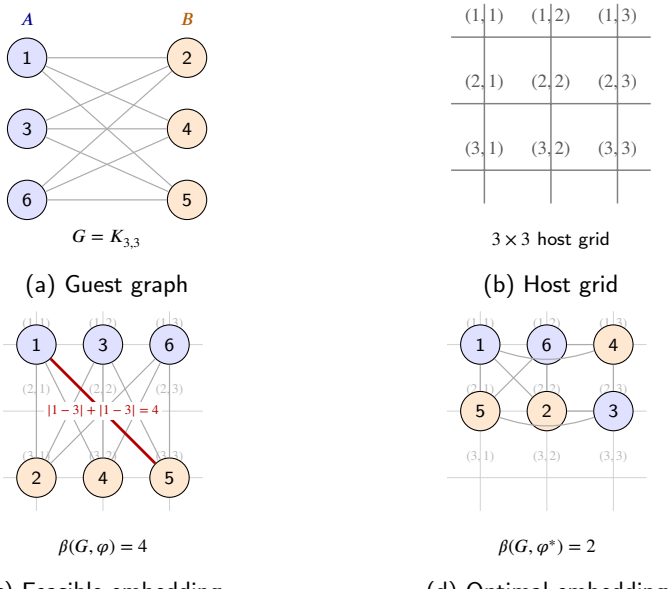


Figure 1: A 2DBMP example for $G = K_{3,3}$ on a 3×3 host grid. The feasible and optimal embeddings have bandwidths 4 and 2, respectively.

The 2DBMP places the vertices of a guest graph at distinct positions on a two-dimensional host grid and minimizes the largest distance between the grid positions of adjacent guest vertices [15]. Here, the guest graph is a simple,

undirected, and unweighted graph $G = (V, E)$ with $n \geq 2$. Its vertex set is $V = \{1, \ldots, n\}$, and E denotes its nonempty edge set. The host grid consists of $h \times h$ positions arranged in h rows and h columns. In the square-root-grid setting considered here, the grid side is $h = \lceil \sqrt{n} \rceil$, following recent 2DBMP studies [3, 23]. Let $\varphi(v) = (x_v, y_v)$ denote the grid position assigned to vertex v. Equation (1) defines the bandwidth of an embedding as the maximum Manhattan distance over all guest edges. The 2DBMP seeks a feasible embedding φ that minimizes $\beta(G, \varphi)$. The minimum bandwidth over all feasible embeddings of G is denoted by $\beta^*(G) = \min_\varphi \beta(G, \varphi)$. Figure 1 illustrates this setting using the guest graph $K_{3,3}$ and a 3×3 host grid. Panels (a) and (b) show the graph and the grid before vertex placement. Panels (c) and (d) compare a feasible embedding with bandwidth 4 and an optimal embedding with bandwidth 2. The highlighted edge in panel (c) has Manhattan distance 4, which is the largest edge distance in this embedding.

$$\beta(G, \varphi) = \max_{(u,v)\in E} \left(|x_u - x_v| + |y_u - y_v| \right) \tag{1}$$

Equations (2)–(10) give an ILP formulation of the 2DBMP. Let $H_h = \{(r, c) : 1 \leq r, c \leq h\}$ denote the host-grid positions. For each $v \in V$ and $(r, c) \in H_h$, let a_{vrc} be a binary placement variable. For each edge $e = (u, v) \in E$, the model uses four nonnegative integer auxiliary variables, $d_e^{x,+}, d_e^{x,-}, d_e^{y,+}, d_e^{y,-}$, to represent row and column coordinate differences without absolute-value terms. It also uses one integer bandwidth variable β.

$$\min \quad \beta \tag{2}$$

$$\text{s.t.} \quad \sum_{r=1}^{h} \sum_{c=1}^{h} a_{vrc} = 1 \qquad \forall v \in V, \tag{3}$$

$$\sum_{v\in V} a_{vrc} \leq 1 \qquad \forall (r, c) \in H_h, \tag{4}$$

$$\left(\sum_{r=1}^{h} \sum_{c=1}^{h} r\, a_{urc} \right) - \left(\sum_{r=1}^{h} \sum_{c=1}^{h} r\, a_{vrc} \right) = d_e^{x,+} - d_e^{x,-} \qquad \forall e = (u, v) \in E, \tag{5}$$

$$\left(\sum_{r=1}^{h} \sum_{c=1}^{h} c\, a_{urc} \right) - \left(\sum_{r=1}^{h} \sum_{c=1}^{h} c\, a_{vrc} \right) = d_e^{y,+} - d_e^{y,-} \qquad \forall e = (u, v) \in E, \tag{6}$$

$$d_e^{x,+} + d_e^{x,-} + d_e^{y,+} + d_e^{y,-} \leq \beta \qquad \forall e \in E, \tag{7}$$

$$a_{vrc} \in \{0, 1\} \qquad \forall v \in V,\ (r, c) \in H_h, \tag{8}$$

$$d_e^{x,+}, d_e^{x,-}, d_e^{y,+}, d_e^{y,-} \in \{0, \ldots, h - 1\} \qquad \forall e \in E, \tag{9}$$

$$\beta \in \mathbb{Z}_{\geq 0}. \tag{10}$$

Constraints (3) and (4) assign each vertex to exactly one host-grid position and allow each position to contain at most one vertex, so they enforce an injective placement. For each vertex $v \in V$, let $x_v = \sum_{r,c} r\, a_{vrc}$ and $y_v = \sum_{r,c} c\, a_{vrc}$, where the sums range over $(r, c) \in H_h$. These expressions are the row and column coordinates induced by the placement variables. For an edge $e = (u, v)$, Constraints (5)– (7) linearize the condition $|x_u - x_v| + |y_u - y_v| \leq \beta$. The split variables are nonnegative, and Constraints (5) and (6) write the signed row and column differences as differences of such variables. Lemma 1 in Appendix A formalizes why this represents the Manhattan distance without using absolute-value terms. Constraint (7) bounds the resulting edge distance by β for every guest edge. Finally, Constraints (8)–(10) specify the variable domains, and minimizing β in (2) is therefore equivalent to minimizing the maximum Manhattan distance over all guest edges.

2.2. Related Work

Research on 2DBMP comprises theoretical analyses and computational approaches based on heuristic search, exact formulations, or combinations of both [3, 4, 15, 23]. These studies are not always directly comparable because they may adopt different distance metrics or host-grid dimensions [4, 13, 15]. Accordingly, the discussion below distinguishes the standard square-root-grid L_1 2DBMP considered in this paper from L_∞ formulations and alternative host grids.

Theoretical studies established bounds and exact values for selected graph classes under broader host-grid formulations. For the L_1 model, Lin [17] derived density-based lower bounds and a general upper bound based on the minimum L_1-diameter of n grid points. The corresponding L_∞ formulation and exact results for several graph classes were developed by Lin et al. [18]. The L_1 and L_∞ models were subsequently studied within a common framework

by Lin and Lin [15], while Lin and Lin [16] investigated the square-root relationship between one-dimensional and two-dimensional bandwidth. These studies use $n \times n$ or otherwise sufficiently large host grids rather than the compact $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host used for the benchmark setting considered in this paper. Their results provide theoretical foundations for 2DBMP, but they do not directly establish optimal values for the compact-grid benchmark instances.

Heuristic methods have been used to construct feasible embeddings and obtain strong upper bounds without requiring an optimality proof. For the L_1 problem, Rodríguez-García et al. [22] proposed a GRASP-based approach for square-grid hosts. Subsequent methods for the compact square-root-grid setting include the BVNS heuristic developed alongside the exact constraint satisfaction problem (CSP) models of Rodríguez-García et al. [23], iterated greedy with local search [3], dual-representation simulated annealing [25], and reinforcement-learning-driven tabu search [26]. Related L_1 studies considered simulated annealing on a $2 \times \lceil n/2 \rceil$ host [13] and RVNS on balanced rectangular hosts [14]. For the L_∞ variant, Cavero et al. [4] combined a multi-start heuristic with exact CSP formulations. Although these heuristic methods can produce high-quality feasible embeddings, the resulting bandwidths remain upper bounds or best-known values unless an exact method separately certifies optimality.

Exact methods complement heuristic search by seeking both a feasible embedding and a certificate that no embedding with a smaller bandwidth exists. Within this line of research, Rodríguez-García et al. [23] developed CSP models for the standard L_1 2DBMP and evaluated them on established benchmark instances. More recently, Cavero et al. [4] developed CSP models for the L_∞ variant of 2DBMP, in which edge lengths are measured using Chebyshev distance rather than the L_1 Manhattan distance considered in this study. To reduce the range of bandwidth values examined by the exact search, their procedure uses the maximum vertex degree to identify a lower bound below which no feasible embedding can exist. This lower bound is obtained by counting the distinct grid positions available around a vertex within each Chebyshev distance. These studies establish CSP as an exact approach for both distance metrics. However, L_1 and L_∞ define different optimization objectives even on the same graph instances, so optimality under Chebyshev distance does not certify optimality under Manhattan distance. The reported exact results for the L_1 setting also leave many larger benchmark instances unresolved [23], motivating the investigation of alternative exact formulations.

Overall, prior work provides theoretical bounds, heuristic upper bounds, and exact methods. However, there is still a gap between obtaining high-quality feasible embeddings and certifying their optimality in the standard square-root-grid Manhattan-distance setting. This limitation motivates the alternative SAT-based exact formulation developed in the next section.

3. SAT-Based Exact Approach

3.1. SAT Encoding Overview

The SAT approach reduces the 2DBMP to a sequence of decision problems, each associated with a bandwidth threshold K. For a fixed K, the method constructs a SAT formula representing the existence of a feasible embedding φ such that $\beta(G, \varphi) \leq K$. After obtaining a feasible embedding, the method sets the next threshold K to one less than the bandwidth of that embedding. If the resulting formula is unsatisfiable, no feasible embedding with a smaller bandwidth exists. Therefore, the bandwidth of the current embedding is the optimal value $\beta^*(G)$.

The 2DBMP constraints are encoded by the position, occupancy, distance, and bandwidth components of the SAT formula in Equation (11), while the optional symmetry component is added to reduce the search space. The position component $\mathcal{F}_{\text{pos}}$ assigns one row and one column to each vertex, while the occupancy component $\mathcal{F}_{\text{occ}}^{\text{seq}}$ prevents two vertices from occupying the same grid position. The distance component $\mathcal{F}_{\text{dist}}^{U^*}$ represents coordinate separations up to the cutoff U^*. The symmetry component $\mathcal{F}_{\text{sym}}$ removes a safe subset of symmetry-equivalent placements by restricting a selected anchor vertex. The bandwidth component Δ_K contains the clauses that enforce the current bandwidth threshold. Thus, when K decreases, the position, occupancy, distance, and symmetry components remain unchanged, while only the bandwidth component Δ_K changes.

$$\mathcal{F}_{\text{full}}(K) = \mathcal{F}_{\text{pos}} \wedge \mathcal{F}_{\text{occ}}^{\text{seq}} \wedge \mathcal{F}_{\text{dist}}^{U^*} \wedge \mathcal{F}_{\text{sym}} \wedge \Delta_K \tag{11}$$

Among the component roles in Equation (11), position, occupancy, distance, and bandwidth enforcement are required to represent a bandwidth-K embedding, whereas symmetry breaking is an optional search optimization. The symmetry component $\mathcal{F}_{\text{sym}}$ can therefore be removed without changing which bandwidth values are feasible. The required occupancy component can be represented by the sequential occupancy encoding $\mathcal{F}_{\text{occ}}^{\text{seq}}$, which uses cell-indicator variables and sequential-counter constraints, or by the pairwise occupancy encoding $\mathcal{F}_{\text{occ}}^{\text{pair}}$. Similarly, the

required distance component can use the cutoff-aware form $\mathcal{F}_{\text{dist}}^{U^*}$ or the full form $\mathcal{F}_{\text{dist}}$. The cutoff-aware form preserves exactness only when U^* is valid. The required bandwidth component Δ_K is shared by both solving schemes. The incremental scheme retains the solver state as K decreases, whereas the non-incremental scheme rebuilds the solver for each value of K.

To construct the SAT formula, the method uses several types of Boolean variables. For each vertex $v \in V$ and each coordinate $i \in \{1, \dots, h\}$, the Boolean variables $X_{v,i}$ and $Y_{v,i}$ encode the row and column choices of v. For each vertex $v \in V$ and each grid position $(r, c) \in H_h$, the Boolean variable $A_{v,r,c}$ indicates that v occupies this position. For each edge $e = (u, v)$, each axis $q \in \{x, y\}$, and each represented threshold d, the Boolean variable $T_{e,d}^{q}$ indicates that the coordinate separation of u and v along axis q is at least d. The following subsections present these components, their alternatives where applicable, and the conditions required for the optional restrictions to preserve correctness.

3.2. Position and Occupancy Encoding

The position component $\mathcal{F}_{\text{pos}}$ corresponds to the one-position-per-vertex condition in Constraint (3) of the ILP model. In the SAT encoding, this component requires each vertex to select exactly one row and exactly one column. Equations (12) and (13) state the corresponding cardinality constraints for the row and column variables, respectively. These two constraints are encoded into CNF using the Sequential Counter encoding of Sinz [24], as implemented in PySAT [11, 12].

$$\sum_{i=1}^{h} X_{v,i} = 1 \qquad \forall v \in V \tag{12}$$

$$\sum_{i=1}^{h} Y_{v,i} = 1 \qquad \forall v \in V \tag{13}$$

The occupancy component corresponds to Constraint (4) in the ILP model, which requires each grid position to contain at most one vertex. A direct way to enforce this condition is to consider every grid position $(r, c) \in H_h$ and every pair of distinct vertices $u, v \in V$, and forbid both vertices from occupying that position simultaneously. This construction applies the standard pairwise encoding of an at-most-one constraint to each grid position [21]. Equation (14) defines the resulting pairwise occupancy encoding $\mathcal{F}_{\text{occ}}^{\text{pair}}$. Each clause in Equation (14) is violated exactly when both u and v occupy (r, c), so the conjunction ensures that every grid position contains at most one vertex.

$$\mathcal{F}_{\text{occ}}^{\text{pair}} = \bigwedge_{(r,c)\in H_h} \bigwedge_{\substack{u,v\in V \\ u<v}} \left(\neg X_{u,r} \vee \neg Y_{u,c} \vee \neg X_{v,r} \vee \neg Y_{v,c}\right) \tag{14}$$

For a graph with n vertices, the encoding introduces one clause for every unordered pair of distinct vertices at each grid position, resulting in $\binom{n}{2}$ clauses per position. Since the host grid has $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ positions, the complete pairwise occupancy encoding contains $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil \times \binom{n}{2}$ clauses. Thus, the pairwise occupancy encoding contains $\Theta(n^3)$ clauses in the square-root-grid setting. Instead of constructing the occupancy condition from pairwise clauses, the proposed method formulates an at-most-one cardinality constraint for each grid position. Specifically, for each vertex $v \in V$ and grid position $(r, c) \in H_h$, the method introduces a Boolean variable $A_{v,r,c}$ indicating whether v occupies (r, c). To enforce this meaning, Equations (15)–(17) link $A_{v,r,c}$ to the row and column variables.

$$\neg A_{v,r,c} \vee X_{v,r} \qquad \forall v \in V,\ (r, c) \in H_h \tag{15}$$

$$\neg A_{v,r,c} \vee Y_{v,c} \qquad \forall v \in V,\ (r, c) \in H_h \tag{16}$$

$$A_{v,r,c} \vee \neg X_{v,r} \vee \neg Y_{v,c} \qquad \forall v \in V,\ (r, c) \in H_h \tag{17}$$

For each grid position $(r, c) \in H_h$, the requirement that no more than one vertex occupies that position is expressed by applying an at-most-one constraint to the variables $\{A_{v,r,c} : v \in V\}$, as stated in Equation (18). This constraint can be translated into CNF using different cardinality encodings without changing the represented occupancy condition. In this study, these constraints are encoded using the Sequential Counter encoding of Sinz [24], as implemented in PySAT [11, 12]. Together, the linking clauses in Equations (15)–(17) and the Sequential Counter encoding of Equation (18) define the sequential occupancy encoding $\mathcal{F}_{\text{occ}}^{\text{seq}}$. For the n occupancy variables associated with each

grid position, this encoding introduces $\Theta(n)$ clauses and $\Theta(n)$ auxiliary variables. Across the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ grid positions, the resulting counts are therefore $\Theta(n^2)$ clauses and $\Theta(n^2)$ auxiliary variables. After the variables $A_{v,r,c}$ and their linking clauses are included, the complete sequential occupancy encoding remains $\Theta(n^2)$ in both clauses and auxiliary variables, whereas the pairwise occupancy encoding contains $\Theta(n^3)$ clauses. Despite their different CNF representations, $\mathcal{F}_{\text{occ}}^{\text{seq}}$ and $\mathcal{F}_{\text{occ}}^{\text{pair}}$ enforce the same occupancy condition over the placement variables.

$$\sum_{v \in V} A_{v,r,c} \leq 1 \qquad \forall (r,c) \in H_h \tag{18}$$

3.3. Distance Encoding

The distance component corresponds to Constraints (5)– (7) in the ILP model, which linearize the absolute-value condition for the Manhattan distance of each guest edge. In the full SAT encoding, these differences are represented by Boolean threshold variables. For each edge $e = (u, v) \in E$, coordinate axis $q \in \{x, y\}$, where $q = x$ denotes the row axis and $q = y$ denotes the column axis, and threshold $d \in \{1, \ldots, h-1\}$, the variable $T_{e,d}^{q}$ is true exactly when the coordinate separation of u and v along axis q is at least d. To express the corresponding clauses uniformly, the method uses $Z_{w,i}^{q}$ as shorthand for $X_{w,i}$ when $q = x$ and $Y_{w,i}$ when $q = y$, where $w \in V$ and $i \in \{1, \ldots, h\}$.

Equations (19)–(24) enforce the threshold semantics of $T_{e,d}^{q}$. When the selected coordinates are exactly d positions apart, Equations (19) and (20) force $T_{e,d}^{q}$ to be true for the two possible coordinate orders. Equations (21) and (22) encode the upper boundary of an exact separation. If the selected coordinates are exactly d positions apart, the separation cannot be at least $d + 1$, so these equations force $T_{e,d+1}^{q}$ to be false for the two possible coordinate orders. Equation (23) handles zero separation along one axis. If u and v select the same row for $q = x$, or the same column for $q = y$, then $T_{e,1}^{q}$ is forced to be false. Equation (24) enforces consistency among the threshold variables. If the coordinate separation is at least $d + 1$, it must also be at least d, so $T_{e,d+1}^{q}$ implies $T_{e,d}^{q}$. Together with the unique coordinate assignments enforced by $\mathcal{F}_{\text{pos}}$, these clauses establish the intended threshold semantics. Specifically, $T_{e,d}^{q}$ is true if and only if the coordinate separation of u and v is at least d. The conjunction of these clauses over all $e = (u, v) \in E$, $q \in \{x, y\}$, and the stated ranges of d and k is denoted by $\mathcal{F}_{\text{dist}}$.

$$Z_{v,k}^{q} \wedge Z_{u,k-d}^{q} \rightarrow T_{e,d}^{q} \qquad 1 \leq d \leq h-1,\ d+1 \leq k \leq h \tag{19}$$

$$Z_{u,k}^{q} \wedge Z_{v,k-d}^{q} \rightarrow T_{e,d}^{q} \qquad 1 \leq d \leq h-1,\ d+1 \leq k \leq h \tag{20}$$

$$Z_{v,k}^{q} \wedge Z_{u,k-d}^{q} \rightarrow \neg T_{e,d+1}^{q} \qquad 1 \leq d \leq h-2,\ d+1 \leq k \leq h \tag{21}$$

$$Z_{u,k}^{q} \wedge Z_{v,k-d}^{q} \rightarrow \neg T_{e,d+1}^{q} \qquad 1 \leq d \leq h-2,\ d+1 \leq k \leq h \tag{22}$$

$$Z_{u,k}^{q} \wedge Z_{v,k}^{q} \rightarrow \neg T_{e,1}^{q} \qquad 1 \leq k \leq h \tag{23}$$

$$T_{e,d+1}^{q} \rightarrow T_{e,d}^{q} \qquad 1 \leq d \leq h-2 \tag{24}$$

The cutoff-aware encoding $\mathcal{F}_{\text{dist}}^{U^*}$ uses a valid initial bandwidth upper bound U^* to limit the represented axis-distance thresholds, and the descending search starts from the same value. Lin and Lin showed that $\delta(n)$ is the minimum possible maximum Manhattan distance among n distinct lattice points with integer coordinates [15]. However, their result does not guarantee that a configuration attaining $\delta(n)$ fits within the compact $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host used in this study. Let U_{DRSA} denote the bandwidth of a feasible embedding on this compact host returned by DRSA [25]. Since $U^* = \max\{\delta(n), U_{\text{DRSA}}\} \geq U_{\text{DRSA}}$, the known DRSA embedding is feasible at the initial threshold U^*. When $\delta(n) > U_{\text{DRSA}}$, the first SAT call uses $K = \delta(n)$. This initialization is intended to avoid starting the incremental search directly at a tighter bandwidth threshold. Additionally, if U_{DRSA} is unavailable, the bandwidth of another feasible embedding or, for the $h \times h$ host considered here, the host-diameter upper bound $2(h-1)$ can be used instead. Equations (25) and (26) formally define $\delta(n)$ and U^*, respectively.

$$\delta(n) = \min\left\{ 2\left\lceil \frac{\sqrt{2n-1}-1}{2} \right\rceil,\ 2\left\lceil \sqrt{\frac{n}{2}} \right\rceil - 1 \right\} \tag{25}$$

$$U^* = \max\{\delta(n), U_{\text{DRSA}}\} \tag{26}$$

Using this bound, $\mathcal{F}_{\text{dist}}^{U^*}$ restricts the threshold index range to $d \in \{1, \ldots, \min\{U^*, h-1\}\}$ and adds the clauses in Equation (27) to exclude axis separations greater than U^*. By contrast, $\mathcal{F}_{\text{dist}}$ represents the full threshold range

$d \in \{1, \ldots, h-1\}$ and does not include the cutoff clauses. Since the separation along each axis is at most $h-1$, the Manhattan distance of an edge is at most $2(h-1)$. The descending search with $\mathcal{F}_{\text{dist}}$ therefore starts from the initial bandwidth threshold $K = 2(h-1)$. Every embedding with bandwidth at most U^* also has axis separations at most U^*, so the cutoff preserves every embedding within the searched bandwidth range and does not affect the exactness of the encoding.

$$\neg Z^q_{u,i} \vee \neg Z^q_{v,k} \qquad \begin{matrix} e=(u,v)\in E,\ q\in\{x,y\}, \\ i,k\in\{1,\ldots,h\},\ |i-k|>U^* \end{matrix} \tag{27}$$

3.4. Q1 Anchor Symmetry Breaking

Reflecting an embedding horizontally or vertically changes the grid positions of its vertices but preserves all Manhattan edge distances. Following the quarter-grid symmetry-breaking strategy of Rodríguez-García et al. [23], the SAT formulation restricts a deterministically selected anchor vertex a to one quarter of the host grid. The anchor is chosen by maximum degree, then by the maximum sum of the degrees of its neighbors, and finally by the smallest vertex index if a tie remains.

Define $Q_1 = \{(r,c) \in H_h : 1 \le r, c \le \lceil h/2 \rceil\}$ as the region allowed for the anchor vertex a. Figure 2 illustrates this definition for a 4×4 host grid, where Q_1 is the shaded 2×2 region. To restrict a to Q_1, the SAT formulation forbids every row and column coordinate greater than $\lceil h/2 \rceil$. Using the position variables $X_{a,p}$ and $Y_{a,p}$, Equations (28) and (29) encode these row and column exclusions, respectively, as unit clauses. Together, these unit clauses force both coordinates of a to lie in $\{1, \ldots, \lceil h/2 \rceil\}$, and therefore restrict a to Q_1. The conjunction of the unit clauses in Equations (28) and (29) is denoted by $\mathcal{F}_{\text{sym}}$.

$$\neg X_{a,p} \qquad \forall p \in \left\{ \left\lceil \frac{h}{2} \right\rceil + 1, \ldots, h \right\} \tag{28}$$

$$\neg Y_{a,p} \qquad \forall p \in \left\{ \left\lceil \frac{h}{2} \right\rceil + 1, \ldots, h \right\} \tag{29}$$

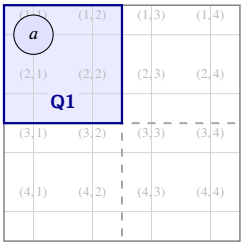


Figure 2: Q1 region used for anchor symmetry breaking on a 4×4 host grid. The dashed reflection axes divide the grid into four 2×2 regions. The anchor vertex a is restricted to the shaded Q1 region.

For any feasible embedding, horizontal and vertical reflections can be applied independently so that the anchor vertex lies in Q_1. Each reflection is a bijection of H_h and preserves Manhattan distances, so the transformed embedding remains injective and has the same bandwidth. Consequently, adding $\mathcal{F}_{\text{sym}}$ does not change whether a bandwidth-K embedding exists and does not change the optimal value. Lemma 5 in Appendix B provides the formal preservation proof.

3.5. Incremental Solving and Certification

The bandwidth component Δ_K is the only component that depends on K. It enforces the bound K through clauses constructed from the threshold variables of the selected distance encoding. Equations (30a) and (30b) define Δ_K for $\mathcal{F}^{U^*}_{\text{dist}}$ and $\mathcal{F}_{\text{dist}}$, respectively. Both forms contain the same two clause groups. Their only difference is that the largest encoded threshold is $\min\{U^*, h-1\}$ in the cutoff-aware form and $h-1$ in the full-distance form. The first group contains a unit clause for each edge and axis when the threshold at $K+1$ does not exceed the applicable limit. This clause restricts the separation along that axis to at most K. The second group considers every decomposition $K+1 = i + (K-i+1)$ whose two terms do not exceed the applicable limit and forbids the two axis separations from

reaching these thresholds simultaneously. Because the two thresholds sum to $K+1$, every forbidden combination would give the edge a Manhattan distance greater than K.

$$\Delta_K = \bigwedge_{e\in E}\left(\bigwedge_{\substack{q\in\{x,y\}\\ K+1\le\min\{U^*,h-1\}}} \neg T^q_{e,K+1} \wedge \bigwedge_{\substack{1\le i\le K\\ i\le\min\{U^*,h-1\},\ K-i+1\le\min\{U^*,h-1\}}} \left(\neg T^x_{e,i} \vee \neg T^y_{e,K-i+1}\right)\right) \tag{30a}$$

$$\Delta_K = \bigwedge_{e\in E}\left(\bigwedge_{\substack{q\in\{x,y\}\\ K+1\le h-1}} \neg T^q_{e,K+1} \wedge \bigwedge_{\substack{1\le i\le K\\ i\le h-1,\ K-i+1\le h-1}} \left(\neg T^x_{e,i} \vee \neg T^y_{e,K-i+1}\right)\right) \tag{30b}$$

Table 1 summarizes the four complete encoding configurations considered in this study. The proposed encoding is denoted by $\mathcal{F}_{\text{full}}(K)$. The remaining notation identifies the component changed relative to the proposed encoding. The notation $\mathcal{F}_{\text{no-sym}}(K)$ removes Q1 anchor symmetry breaking. The notation $\mathcal{F}_{\text{pair}}(K)$ uses pairwise occupancy, while $\mathcal{F}_{\text{full-dist}}(K)$ uses the full distance encoding without the cutoff. All four configurations encode the same bandwidth-K decision problem. In $\mathcal{F}_{\text{full-dist}}(K)$, Δ_K is defined by Equation (30b), and the descending search starts from $K = 2(h-1)$, the maximum possible Manhattan distance between two positions on an $h\times h$ grid. In configurations containing $\mathcal{F}^{U^*}_{\text{dist}}$, Δ_K is defined by Equation (30a), and the search starts from $K = U^*$, the initial bandwidth upper bound used to construct $\mathcal{F}^{U^*}_{\text{dist}}$.

Table 1
Complete SAT encoding configurations considered in this study.

Notation	Definition	Description
$\mathcal{F}_{\text{full}}(K)$	$\mathcal{F}_{\text{pos}} \wedge \mathcal{F}^{\text{seq}}_{\text{occ}} \wedge \mathcal{F}^{U^*}_{\text{dist}} \wedge \mathcal{F}_{\text{sym}} \wedge \Delta_K$	Proposed encoding
$\mathcal{F}_{\text{no-sym}}(K)$	$\mathcal{F}_{\text{pos}} \wedge \mathcal{F}^{\text{seq}}_{\text{occ}} \wedge \mathcal{F}^{U^*}_{\text{dist}} \wedge \Delta_K$	Proposed encoding without Q1 anchor symmetry breaking
$\mathcal{F}_{\text{pair}}(K)$	$\mathcal{F}_{\text{pos}} \wedge \mathcal{F}^{\text{pair}}_{\text{occ}} \wedge \mathcal{F}^{U^*}_{\text{dist}} \wedge \mathcal{F}_{\text{sym}} \wedge \Delta_K$	Proposed encoding with pairwise occupancy in place of sequential occupancy
$\mathcal{F}_{\text{full-dist}}(K)$	$\mathcal{F}_{\text{pos}} \wedge \mathcal{F}^{\text{seq}}_{\text{occ}} \wedge \mathcal{F}_{\text{dist}} \wedge \mathcal{F}_{\text{sym}} \wedge \Delta_K$	Proposed encoding without the cutoff-aware distance restriction

The incremental procedure applies to any encoding configuration in Table 1 because Δ_K is the only component that changes with K. For the selected configuration, $fixedClauses$ denotes the clauses generated from all remaining components, which are added to the SAT solver once before the threshold search begins. For example, when $\mathcal{F}_{\text{full}}(K)$ is selected, $fixedClauses$ contains the clauses generated from $\mathcal{F}_{\text{pos}}$, $\mathcal{F}^{\text{seq}}_{\text{occ}}$, $\mathcal{F}^{U^*}_{\text{dist}}$, and $\mathcal{F}_{\text{sym}}$. At each tested value of K, $thresholdClauses$ denotes the clauses generated from Δ_K, which are added before the corresponding solver call. Table 2 summarizes the complete procedure, including the update of K from the actual bandwidth computed from a satisfiable model and termination with status OPTIMAL, FEASIBLE, or TIMEOUT. Here, OPTIMAL indicates that the returned $bestBandwidth$ is a certified optimal value. Status FEASIBLE indicates that the search timed out after finding a feasible upper bound, whereas TIMEOUT indicates that it timed out without finding a feasible embedding.

When the solver returns SAT at threshold K, the decoded model yields a feasible embedding φ whose actual bandwidth $\beta(G,\varphi)$ is at most K. Because this value may be smaller than K, the procedure sets the next threshold to $actualBandwidth - 1$, thereby skipping intermediate thresholds that cannot improve the current feasible upper bound. If the formula at this next threshold is UNSAT, no embedding with bandwidth below $actualBandwidth$ exists. Together with the feasible embedding already found, this establishes $actualBandwidth$ as a certified optimal value. If $actualBandwidth = 1$, it already attains the trivial lower bound for any graph containing at least one edge and can therefore be certified without another solver call. A timeout establishes neither feasibility nor infeasibility at the current threshold, so the procedure terminates without an optimality certificate.

As K decreases, each newly added Δ_K strengthens the current formula, while the previously added threshold clauses remain in the same solver. Reusing the solver retains both these clauses and the learned information obtained from earlier calls. The non-incremental ablation instead creates a fresh solver, adds $fixedClauses$ and the current

thresholdClauses for each tested threshold, and discards that solver after the call. Thus, both procedures solve the same bandwidth-K decision problems but differ in whether clauses and learned information from earlier calls are retained. Appendix B presents the correctness proofs for the encoding configurations in Table 1 and proves that a value returned with status OPTIMAL by the incremental procedure is a certified optimal value.

Table 2
Incremental threshold search and certification.

Input	A graph G, an encoding configuration selected from Table 1, a time limit, and an initial bandwidth upper bound U.
Step 1	For $\mathcal{F}_{\text{full}}(K)$, set $fixedClauses$ to the clauses generated from $\mathcal{F}_{\text{pos}}$, $\mathcal{F}_{\text{occ}}^{\text{seq}}$, $\mathcal{F}_{\text{dist}}^{U^*}$, and $\mathcal{F}_{\text{sym}}$. The other configurations in Table 1 are handled analogously, with Δ_K excluded from $fixedClauses$ in every case.
Step 2	Create a SAT solver and add $fixedClauses$ to it. Set $K \leftarrow U$ and $bestBandwidth \leftarrow$ UNDEFINED.
Step 3	**While** $K \geq 1$ **do** Set $thresholdClauses$ to the clauses generated from Δ_K. Add $thresholdClauses$ to the same solver. Set $result \leftarrow$ Solve(). **If** the time limit is reached and $bestBandwidth$ is defined, **return** $bestBandwidth$ with status FEASIBLE. **If** the time limit is reached and $bestBandwidth$ is undefined, **return** status TIMEOUT. **If** $result =$ UNSAT, **return** $bestBandwidth$ with status OPTIMAL. Decode the SAT model into an embedding φ. Set $actualBandwidth \leftarrow \beta(G, \varphi)$. Set $bestBandwidth \leftarrow actualBandwidth$. **If** $bestBandwidth = 1$, **return** $bestBandwidth$ with status OPTIMAL. Set $K \leftarrow bestBandwidth - 1$. **End while**.
Output	The best bandwidth found, if available, together with status OPTIMAL, FEASIBLE, or TIMEOUT.

4. Computational Experiments

4.1. Experimental Setup

Table 3
Summary of the benchmark sets evaluated in this study.

Benchmark set	Instances	n range	$\lvert E\rvert$ range	Host grid
Regular	43	5–21	6–190	$\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$
Harwell–Boeing	93	9–362	50–5856	$\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$
Regular, 2-row	43	5–21	6–190	$2 \times \lceil n/2\rceil$
Harwell–Boeing, 2-row	93	9–362	50–5856	$2 \times \lceil n/2\rceil$
Regular, extended	43	5–21	6–190	$n \times n$
Harwell–Boeing, extended	49	9–147	50–2211	$n \times n$

Table 3 summarizes the benchmark sets evaluated in this study. The Regular set contains 43 guest graphs from the benchmark collection accompanying RLTS [27], which has also been used in earlier 2DBMP studies [3, 23, 26]. The Harwell–Boeing set contains 93 guest graphs derived from sparse-matrix patterns with $n \leq 400$ from the Matrix Market repository [19, 27]. This size limit was imposed to keep the exact experiments within the available computational resources. The same 93 instances were evaluated by DRSA [25], while overlapping subsets were considered in other 2DBMP studies [3, 14, 23, 26]. The remaining columns report the ranges of the numbers of vertices n and edges $|E|$, together with the host-grid dimensions. The standard experiments use the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid. The additional host-grid experiments include a $2 \times \lceil n/2\rceil$ host-grid evaluation on all 43 Regular and 93 Harwell–Boeing instances, and an $n \times n$ host-grid evaluation on all 43 Regular and 49 instances from the same Harwell–Boeing set.

All experiments were conducted on a workstation equipped with an Intel Core i7-12700KF processor and 32 GB of RAM, running Ubuntu 24.04.3 LTS. The SAT implementation invoked CaDiCaL 1.9.5 through the PySAT toolkit [1, 11, 12]. The commercial-solver comparisons used Gurobi Optimizer 12.0.3 [9] and two formulations in IBM ILOG CPLEX Optimization Studio 22.1.1 [10]: a mixed-integer programming formulation (CPLEX MIP) and a constraint-programming formulation (CPLEX CP). A time limit of 3600 s was applied to each local run. For all locally evaluated methods, the reported solving time includes the complete run for each instance, including model or encoding construction and all solver calls.

The initial SAT thresholds and the domains of the commercial bandwidth variable β are specified separately for each host grid. On the standard $h \times h$ grid, where $h = \lceil\sqrt{n}\rceil$, the cutoff-aware SAT configurations use U^* from Equation (26) as both the distance cutoff and the initial threshold. By contrast, $\mathcal{F}_{\text{full-dist}}(K)$ starts from $K = 2(h-1)$, and the commercial solvers (Gurobi, CPLEX MIP, and CPLEX CP) minimize β over $0 \le \beta \le 2(h-1)$. On the $n \times n$ grid, the cutoff-aware SAT configuration uses $\delta(n)$ from Equation (25) as both the distance cutoff and the initial threshold. The commercial solvers minimize β over $0 \le \beta \le 2(n-1)$ on this host. On the $2 \times \lceil n/2 \rceil$ host, $\delta(n)$ is not guaranteed to provide a feasible bandwidth threshold and is therefore used only as the starting value of the initialization procedure. Starting from $\delta(n)$, successive candidate thresholds are generated by increasing the multiplier applied to $\delta(n)$ in steps of 0.5 (yielding $\delta(n), \lceil 1.5\delta(n)\rceil, 2\delta(n), \lceil 2.5\delta(n)\rceil, \ldots$) until the resulting threshold reaches or exceeds the lower bound in Equation (C.4). SAT feasibility checks then determine a valid initial upper bound. Appendix C provides the complete initialization procedure. The resulting upper bound is used as both the distance cutoff and the initial SAT threshold. The commercial solvers minimize β over $0 \le \beta \le \lceil n/2 \rceil$ on this host.

The proposed SAT method, Gurobi, CPLEX MIP, and CPLEX CP were run locally, with each method run once on each instance. The SAT experiments were performed as single runs because preliminary repeated runs on representative instances showed no material variation. Similarly, Gurobi and CPLEX MIP/CP were each run once per instance under their respective default configurations. The two CPLEX formulations are kept separate because they use different optimization models. The third constraint satisfaction programming model (M3) of Rodríguez-García et al. [23] is included as a published exact reference. Published results from BVNS [23], IG [3], RVNS [14], DRSA [25], and RLTS [26] are included only for comparison of objective values and are not treated as evidence of optimality. For the SAT runs, `OPT` is assigned according to Proposition 2, when a feasible bandwidth B is followed by an UNSAT result at $B-1$ or when a feasible bandwidth of 1 is found. For Gurobi and CPLEX, `OPT` denotes the solver's native proven-optimal status. In the summary result tables, `OPT` indicates a certified optimal value, `FEAS` indicates termination at the time limit with a feasible incumbent but without an optimality proof, and `TO` indicates timeout without a feasible solution. In the detailed Appendix tables, a star marks a certified optimal bandwidth, `FEAS` in parentheses marks a feasible incumbent at the time limit, and - `(TO)` marks timeout without a feasible solution. Subsections 4.2.1–4.2.3 then report the standard-grid results on the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host. Specifically, Subsection 4.2.1 evaluates the SAT encoding and incremental solving components, Subsection 4.2.2 compares certified SAT values with published reference values, and Subsection 4.2.3 compares the proposed SAT method with commercial exact solvers. Subsection 4.3 reports the additional host-grid comparisons on the $2 \times \lceil n/2 \rceil$ and $n \times n$ grids.

4.2. Results on the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ Grid

4.2.1. Ablation Study of SAT Encoding and Incremental Solving

Table 4
`OPT` and `FEAS` counts for the five SAT variants on the Regular and Harwell–Boeing sets using the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid.

	Regular (43)		Harwell–Boeing (93)	
Variant	`OPT`	`FEAS`	`OPT`	`FEAS`
$\mathcal{F}_{\text{full}}$	41	2	**42**	**51**
$\mathcal{F}_{\text{no-sym}}$	41	2	34	59
$\mathcal{F}_{\text{pair}}$	41	2	35	58
$\mathcal{F}_{\text{full-dist}}$	41	2	37	56
Non-incremental $\mathcal{F}_{\text{full}}$	41	2	37	56

This ablation compares the proposed $\mathcal{F}_{\text{full}}(K)$ with $\mathcal{F}_{\text{no-sym}}(K)$, $\mathcal{F}_{\text{pair}}(K)$, and $\mathcal{F}_{\text{full-dist}}(K)$, whose configurations are summarized in Table 1. It also includes a non-incremental variant of $\mathcal{F}_{\text{full}}(K)$ to evaluate the effect of reusing

the SAT solver across bandwidth thresholds. Table 4 reports the numbers of instances for which each variant proves optimality or reaches the time limit. On the Regular set, all five variants return the same bandwidth and status on every instance, so no difference in solution quality is observed. On the Harwell–Boeing set, $\mathcal{F}_{\text{full}}$ has the highest certification coverage with 42 instances, compared with 34 for $\mathcal{F}_{\text{no-sym}}$, 35 for $\mathcal{F}_{\text{pair}}$, and 37 for both $\mathcal{F}_{\text{full-dist}}$ and non-incremental $\mathcal{F}_{\text{full}}$. Appendix D reports aggregate variable and clause counts before Δ_K is added.

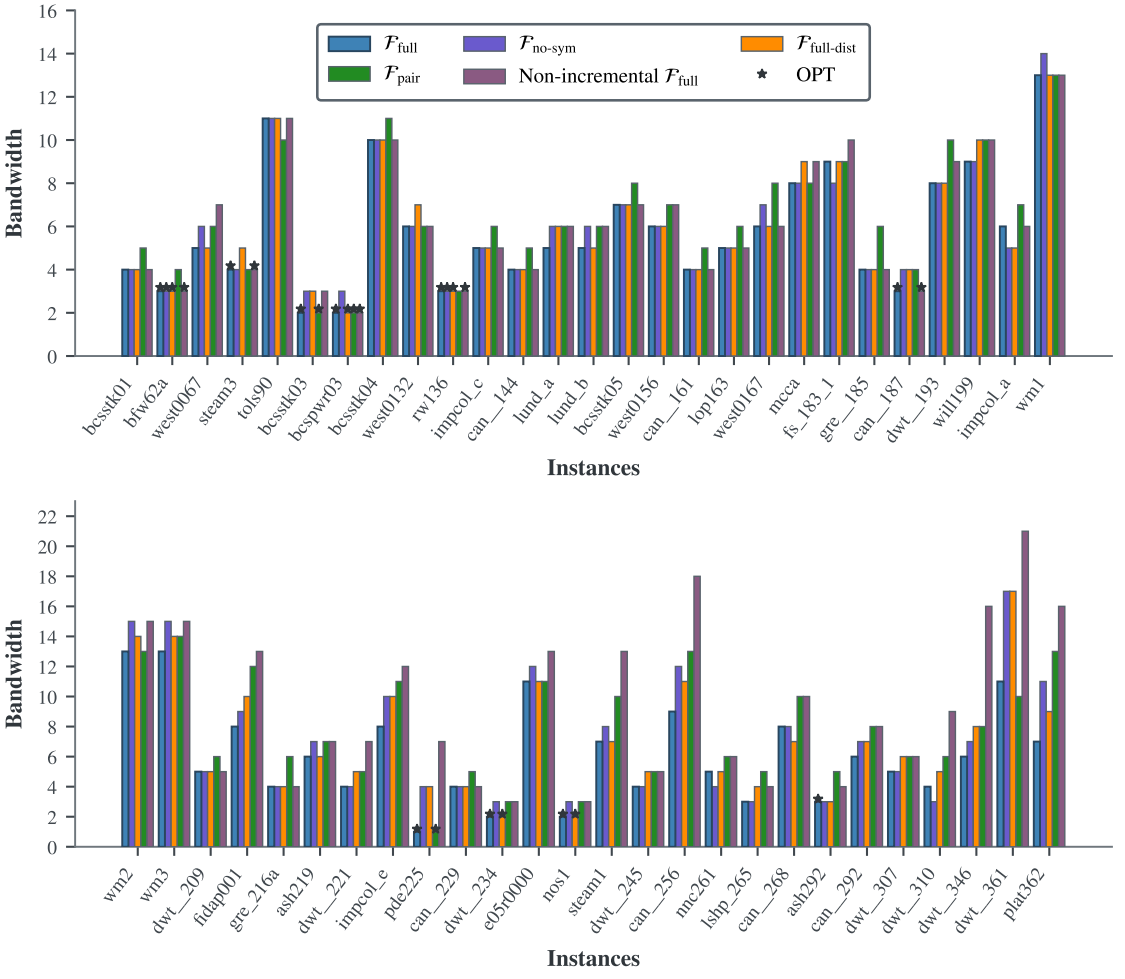


Figure 3: Returned-bandwidth comparison of the five SAT variants on the 53 Harwell–Boeing instances with differing bandwidths or statuses.

Figures 3 and 4 complement Table 4 with instance-level views of returned bandwidths and certification performance, respectively. Figure 3 shows the 53 Harwell–Boeing instances for which at least two variants differ in returned bandwidth or status, whereas Figure 4 plots cumulative solving time over the instances certified by each variant. In this and subsequent bandwidth figures, a star marks a certified optimal bandwidth, an unstarred bar marks a feasible incumbent at the time limit, and - indicates that no feasible bandwidth was found. In Figure 3, $\mathcal{F}_{\text{full}}$ attains the smallest returned bandwidth, including ties, on 46 of the 53 instances. On the remaining seven instances, at least one comparison variant returns a smaller bandwidth. In all seven cases, however, $\mathcal{F}_{\text{full}}$ has status `FEAS`, so these differences concern incumbent quality rather than certified optimal values. Figure 4 shows that $\mathcal{F}_{\text{full}}$ certifies additional Harwell–Boeing instances for which one or more comparison variants reaches the time limit. Over the certification-count range attained by all five variants, however, the cumulative solving-time profiles remain broadly similar. Taken together, these results indicate that the seven weaker `FEAS` incumbents constitute a limited trade-off alongside the higher Harwell–Boeing certification coverage and the generally consistent returned-bandwidth performance of $\mathcal{F}_{\text{full}}$.

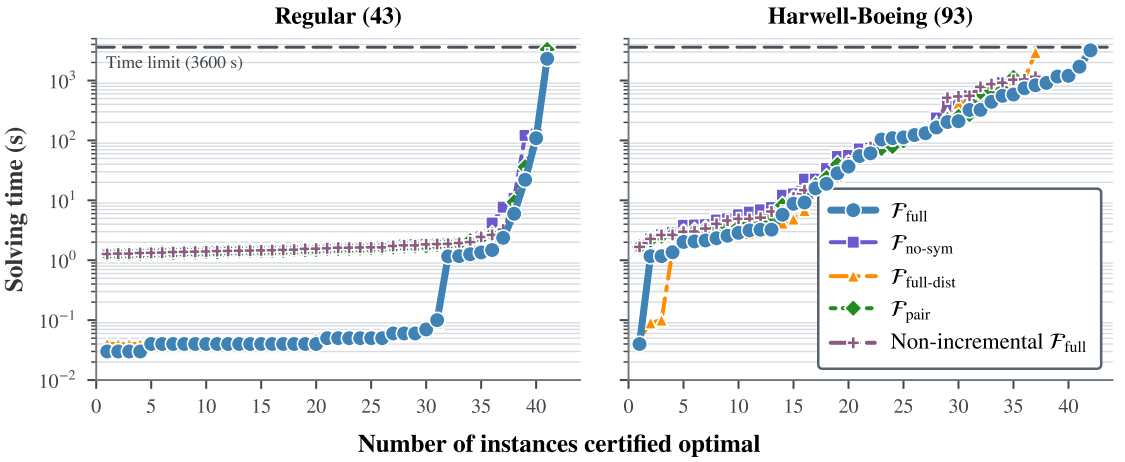


Figure 4: Cumulative solving-time comparison of the five SAT variants on the standard $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid.

4.2.2. Comparison with Published Results

The comparison distinguishes the published exact constraint satisfaction model M3 of Rodríguez-García et al. [23] from the heuristic methods BVNS [23], IG [3], RVNS [14], DRSA [25], and RLTS [26]. The bandwidths published for the heuristic methods are treated as feasible upper bounds rather than evidence of optimality. The published methods were evaluated under different experimental settings. For example, the M3 experiments used a maximum CPU time of 72 h [23], whereas DRSA reported, for each matrix, the best bandwidth obtained from five runs [25]. The comparison in this subsection therefore focuses on published bandwidths and available exact-certification status rather than solving time. Within this audited comparison, $\mathcal{F}_{\text{full}}$ improves the best published bandwidth on three Harwell–Boeing instances: `bfw62a`, `bcsstk03`, and `bcsstk22`. Table 5 reports the corresponding improvements from 4 to 3, 3 to 2, and 3 to 2, all certified optimal by $\mathcal{F}_{\text{full}}$.

Table 5
Certified optimal bandwidths below the compared published values for three Harwell–Boeing instances on the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid.

Instance	n	M3	BVNS	IG	RVNS	DRSA	RLTS	$\mathcal{F}_{\text{full}}$
`bfw62a`	62	–	–	–	–	4	–	**3***
`bcsstk03`	112	–	–	–	–	3	–	**2***
`bcsstk22`	138	8	4	3	4	3	3	**2***

Table 6 places the three improvements within the broader comparison with each published method. For each method, *Published rows* gives the number of instances with an available published bandwidth, and the parenthetical counts for M3 identify how many of those bandwidths were marked optimal in the source study. The *Lower*, *Equal*, and *Higher* columns classify the bandwidth returned by $\mathcal{F}_{\text{full}}$ relative to the corresponding published bandwidth. When a method provides multiple published bandwidths for an instance, the comparison uses its best published bandwidth. The `OPT` block contains comparisons for which $\mathcal{F}_{\text{full}}$ certifies its bandwidth as optimal, whereas the `FEAS` block contains comparisons based on feasible incumbents without an optimality proof.

In comparison with M3, $\mathcal{F}_{\text{full}}$ matches all 39 Regular bandwidths certified by the source study. On Harwell–Boeing, $\mathcal{F}_{\text{full}}$ returns `OPT` for eight of the 24 instances with a published M3 bandwidth, including seven lower bandwidths and one bandwidth equal to the only M3-certified value. The remaining 16 comparisons are `FEAS`, and each incumbent returned by $\mathcal{F}_{\text{full}}$ is lower than the corresponding M3 bandwidth but is not proved optimal. Thus, within this published M3 subset, $\mathcal{F}_{\text{full}}$ extends exact certification from one to eight Harwell–Boeing instances. For the remaining 69 Harwell–Boeing instances without a published M3 bandwidth, $\mathcal{F}_{\text{full}}$ returns a feasible bandwidth in every case and certifies 34 of these values as optimal.

Table 6
Aggregate comparison of $\mathcal{F}_{\text{full}}$ with published bandwidths on the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid, separated into OPT and FEAS groups.

Published method	Published rows	$\mathcal{F}_{\text{full}}$: OPT			$\mathcal{F}_{\text{full}}$: FEAS		
		Lower	Equal	Higher	Lower	Equal	Higher
Regular (43)							
M3	39 (39 OPT)	0	39	0	0	0	0
BVNS	43	6	35	0	0	2	0
IG	43	0	41	0	0	2	0
RLTS	43	0	41	0	0	2	0
Harwell–Boeing (93)							
M3	24 (1 OPT)	7	1	0	16	0	0
BVNS	24	5	3	0	12	2	2
IG	53	10	13	0	0	18	12
RVNS	24	6	2	0	10	2	4
DRSA	93	4	38	0	0	14	37
RLTS	53	6	17	0	0	16	14

In comparison with the heuristic methods, the principal strength of $\mathcal{F}_{\text{full}}$ lies in exact certification rather than uniform dominance in incumbent quality. Every OPT bandwidth returned by $\mathcal{F}_{\text{full}}$ is no higher than the corresponding published upper bound. A lower entry is a method-specific certified improvement, whereas an equal entry certifies the matching heuristic upper bound as optimal. The FEAS comparisons show a more mixed pattern. All six Regular comparisons are equal. On Harwell–Boeing, most FEAS incumbents are lower than or equal to the corresponding BVNS and RVNS values. Equal incumbents occur slightly more often than higher ones for IG and RLTS, while most incumbents are higher than the DRSA values. These results show that $\mathcal{F}_{\text{full}}$ complements heuristic search by certifying matching or improved bandwidths, while its incomplete runs still provide competitive incumbents for several method-specific subsets. The higher FEAS values, particularly relative to DRSA, show that this advantage is not uniform under the 3600 s time limit. Detailed instance-level comparisons are reported in Table E.1 in Appendix E.

4.2.3. Comparison with Commercial Solvers

The proposed $\mathcal{F}_{\text{full}}$ configuration is compared with Gurobi, CPLEX MIP, and CPLEX CP on the same instances and standard square-root host grid, using the same hardware and 3600 s time limit. Table 7 summarizes the certification outcomes. On the Regular set, $\mathcal{F}_{\text{full}}$, Gurobi, and CPLEX MIP certify 41 of 43 instances, while CPLEX CP certifies 42. On the Harwell–Boeing set, the corresponding counts are 42, 10, 8, and 20. The main distinction therefore appears on Harwell–Boeing, where $\mathcal{F}_{\text{full}}$ provides broader exact-certification coverage.

Table 7
OPT and FEAS counts for $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP on the Regular and Harwell–Boeing sets using the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid.

	Regular (43)		Harwell–Boeing (93)	
Method	OPT	FEAS	OPT	FEAS
$\mathcal{F}_{\text{full}}$	41	2	**42**	**51**
Gurobi	41	2	10	83
CPLEX MIP	41	2	8	85
CPLEX CP	**42**	**1**	20	73

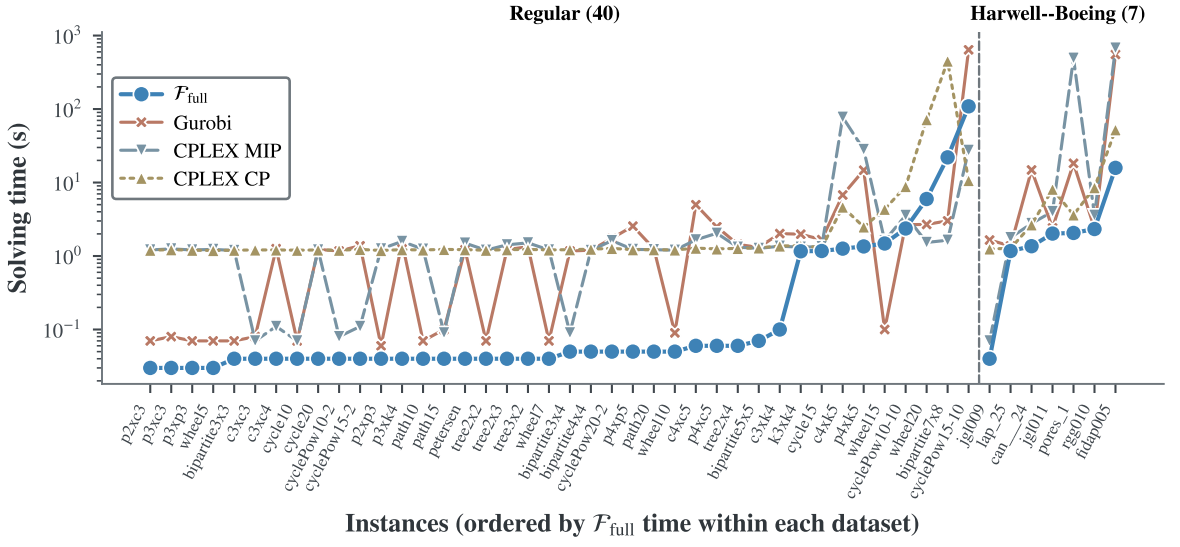


Figure 5: Solving-time comparison of $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP on the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid for the 47 instances on which all four methods certify the same bandwidth.

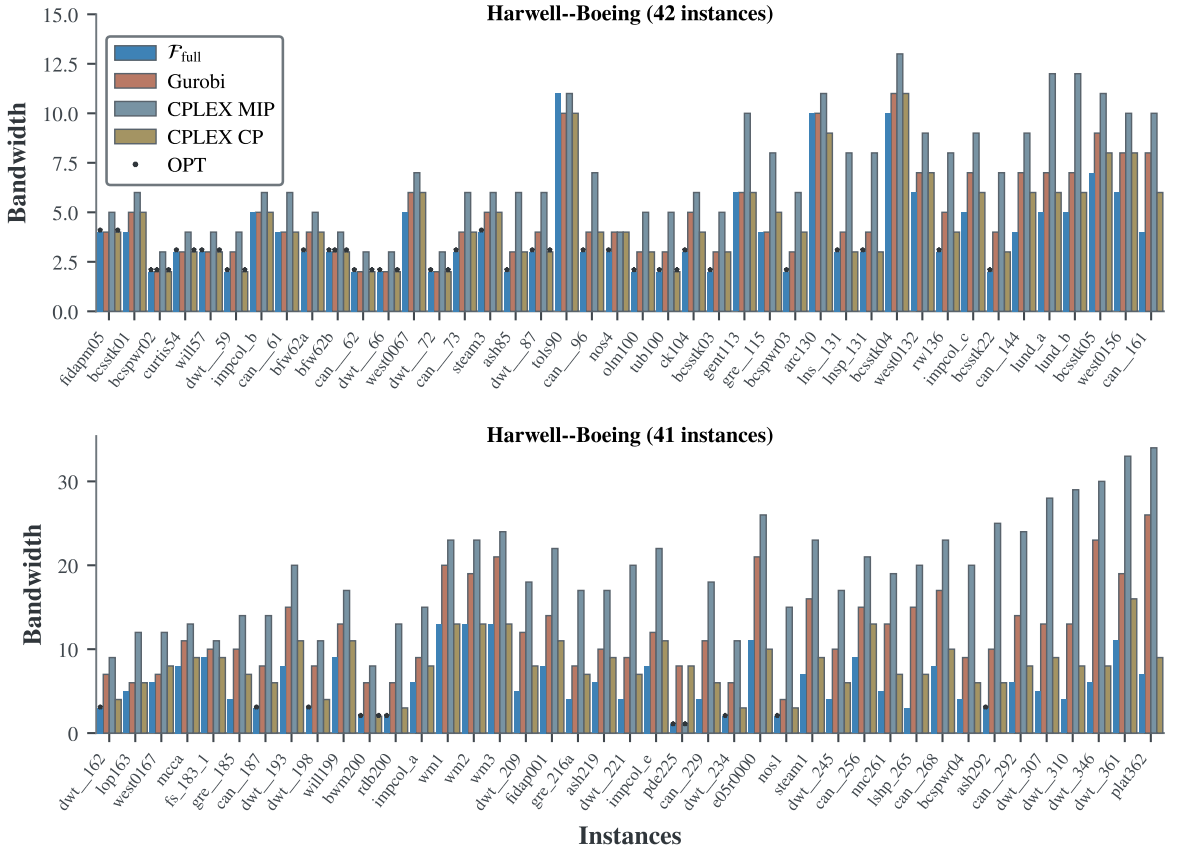


Figure 6: Returned-bandwidth comparison of $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP on the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid for the 83 Harwell–Boeing instances with at least one `FEAS` outcome and differing returned bandwidths.

Figures 5 and 6 complement the certification counts with comparisons of solving time and returned bandwidth, respectively. Figure 6 shows the 83 Harwell–Boeing instances on which at least one method has status FEAS and the returned bandwidths differ. On this subset, $\mathcal{F}_{\text{full}}$ returns a lower bandwidth than Gurobi on 69 instances, CPLEX MIP on 81, and CPLEX CP on 59. Thus, $\mathcal{F}_{\text{full}}$ generally returns lower bandwidths on these timeout-involving instances, although the improvement is not uniform across every comparison. Figure 5 then restricts the solving-time comparison to the 47 instances on which all four methods certify the same bandwidth. On this matched subset, $\mathcal{F}_{\text{full}}$ is faster than Gurobi and CPLEX MIP on 44 of 47 instances and faster than CPLEX CP on 46 of 47. Hence, when both the certified bandwidth and status are matched, $\mathcal{F}_{\text{full}}$ also has a lower solving time in most of the displayed cases.

4.3. Additional Host-Grid Results

4.3.1. Results on the $2 \times \lceil n/2 \rceil$ *Grid*

This experiment compares $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP under the same local experimental setup and 3600 s time limit on the $2 \times \lceil n/2 \rceil$ host grid. Lemma 8 establishes the exactness of the incremental SAT procedure on this host. Table 8 summarizes the outcome counts. On the Regular set, CPLEX MIP certifies all 43 instances, while $\mathcal{F}_{\text{full}}$ and Gurobi certify 42 and CPLEX CP certifies 38. The distinction is clearer on Harwell–Boeing, where $\mathcal{F}_{\text{full}}$ returns OPT on 23 instances, compared with 15 for Gurobi, 7 for CPLEX MIP, and 8 for CPLEX CP. When OPT and FEAS are counted together, the four methods return a feasible bandwidth on 64, 43, 33, and 62 instances, respectively. Thus, $\mathcal{F}_{\text{full}}$ provides the broadest exact-certification coverage and returns a feasible bandwidth on the largest number of Harwell–Boeing instances, although 29 instances remain TO.

Table 8
Summary of results on the Regular and Harwell–Boeing datasets for $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP using the $2 \times \lceil n/2 \rceil$ host grid.

	Regular (43)			Harwell–Boeing (93)		
Method	OPT	FEAS	TO	OPT	FEAS	TO
$\mathcal{F}_{\text{full}}$	42	1	**0**	**23**	41	**29**
Gurobi	42	0	1	15	28	50
CPLEX MIP	**43**	0	**0**	7	26	60
CPLEX CP	38	0	5	8	54	31

Figure 7 restricts the solving-time comparison to the 44 instances on which all four methods return the same bandwidth with status OPT. To avoid conflating runtime with differences in certification outcome, runtime comparisons are also reported on the subset of instances for which all compared methods certify the same bandwidth. On this matched subset, under the reported experimental configuration and single-run protocol, $\mathcal{F}_{\text{full}}$ achieved a lower observed solving time than Gurobi, CPLEX MIP, and CPLEX CP on 41, 38, and 39 instances, respectively. These counts describe the observed single-run results and should not be interpreted as evidence of statistically significant runtime differences. The complete instance-level results for this host grid are reported in Table E.2 in Appendix E.

Figure 8 shows the 66 instances on which at least two methods differ in bandwidth or status. In the pairwise comparisons with Gurobi, CPLEX MIP, and CPLEX CP, $\mathcal{F}_{\text{full}}$ returns a lower bandwidth or returns a bandwidth when the compared solver returns none on 35, 53, and 30 instances, respectively. The reverse occurs on 5, 1, and 11 instances. Thus, $\mathcal{F}_{\text{full}}$ generally provides broader returned-bandwidth coverage and lower incumbents in these comparisons, although the difference is not uniform.

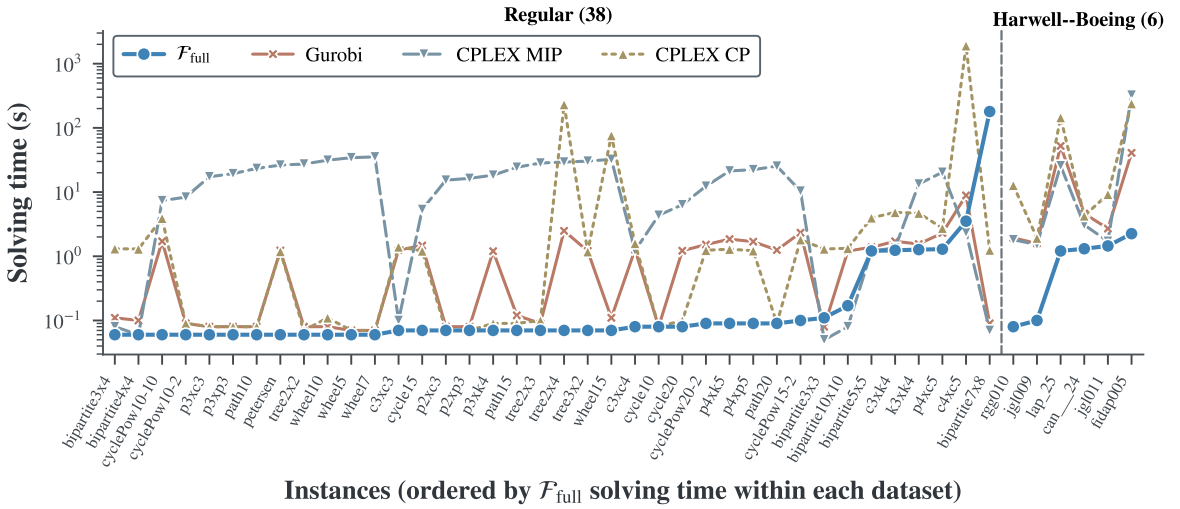


Figure 7: Solving-time comparison of $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP on the $2 \times \lceil n/2 \rceil$ host grid for the 44 instances where all four methods return the same bandwidth with status `OPT`.

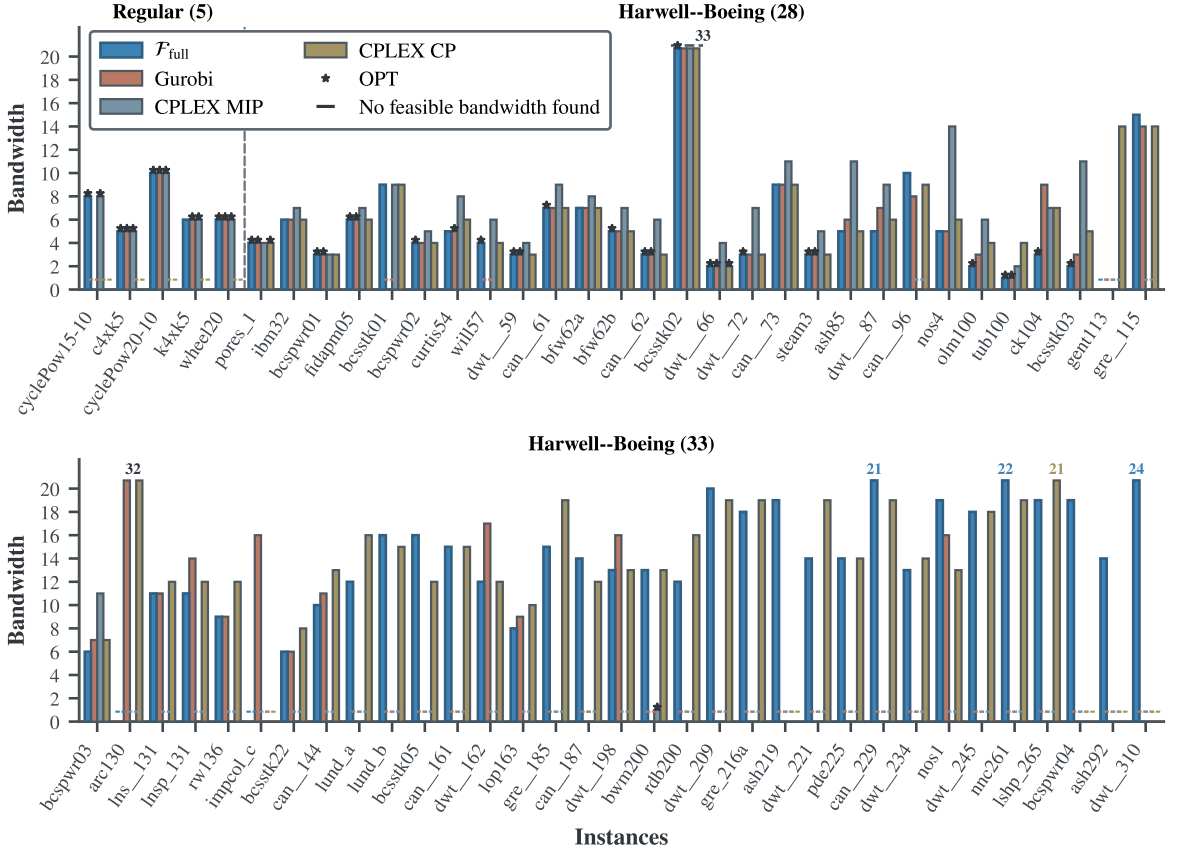


Figure 8: Bandwidth comparison of $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP on the $2 \times \lceil n/2 \rceil$ host grid for the 66 instances where at least two methods differ in bandwidth or status.

4.3.2. Results on the $n \times n$ Grid

These experiments are intended to assess the behavior of the exact SAT approach under alternative host geometries rather than to compare objective values across different host-grid settings. We further extend the matched local comparison of $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP to the $n \times n$ host grid, using the same 3600 s time limit and experimental protocol.

For the Harwell–Boeing benchmark, we restrict this experiment to 49 small-graph instances ($n \leq 150$), as larger instances are substantially more challenging for exact approaches and are difficult to solve or certify within the imposed time limit. This selection provides a more informative comparison under the $n \times n$ host geometry.

Table 9 summarizes the outcomes for 43 Regular and 49 Harwell–Boeing instances. On the Regular set, $\mathcal{F}_{\text{full}}$ certifies 38 instances, compared with 36 for CPLEX CP and 34 each for CPLEX MIP and Gurobi. On Harwell–Boeing, the corresponding counts are 32, 18, 2, and 5. Both $\mathcal{F}_{\text{full}}$ and CPLEX CP return either an `OPT` or `FEAS` outcome on every evaluated instance, whereas CPLEX MIP has 1 Regular and 16 Harwell–Boeing `TO` outcomes and Gurobi has 19 Harwell–Boeing `TO` outcomes. Within this evaluated subset, the main distinction is therefore the broader exact-certification coverage of $\mathcal{F}_{\text{full}}$.

Table 9
Summary of results on the Regular and Harwell–Boeing datasets for $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP using the $n \times n$ host grid.

	Regular (43)			Harwell–Boeing (49)		
Method	`OPT`	`FEAS`	`TO`	`OPT`	`FEAS`	`TO`
$\mathcal{F}_{\text{full}}$	**38**	5	**0**	**32**	17	**0**
CPLEX CP	36	7	**0**	18	31	**0**
CPLEX MIP	34	8	1	2	31	16
Gurobi	34	9	**0**	5	25	19

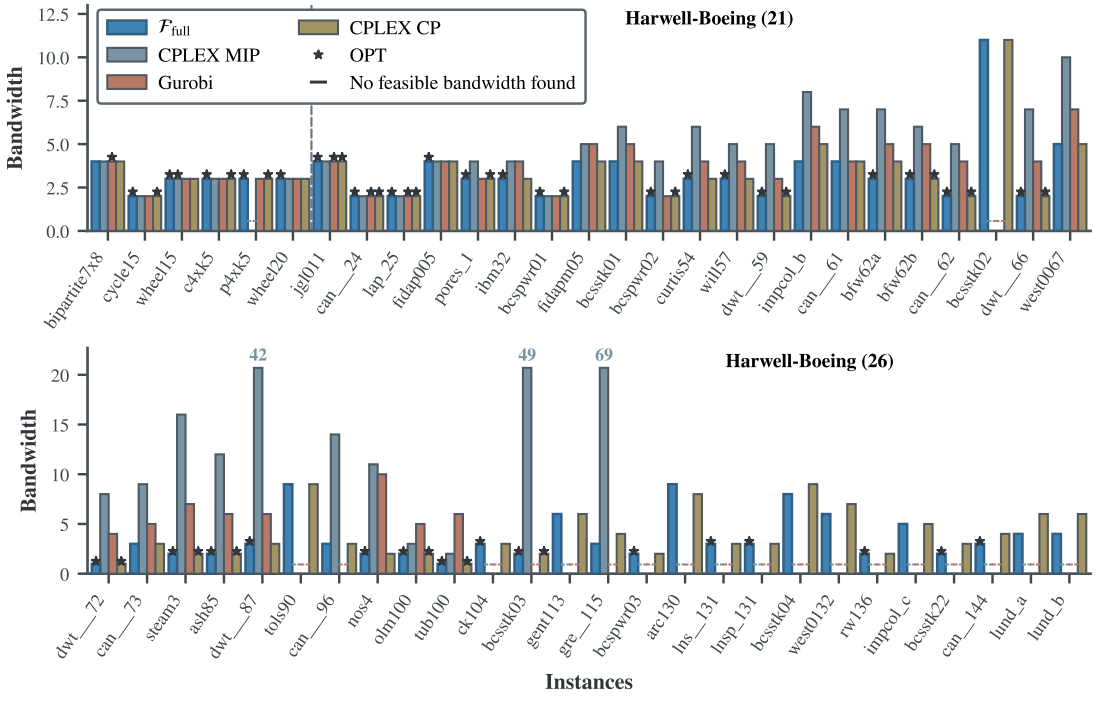


Figure 9: Returned-bandwidth comparison of $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP on the 53 extended $n \times n$ host instances with differing returned bandwidths or statuses.

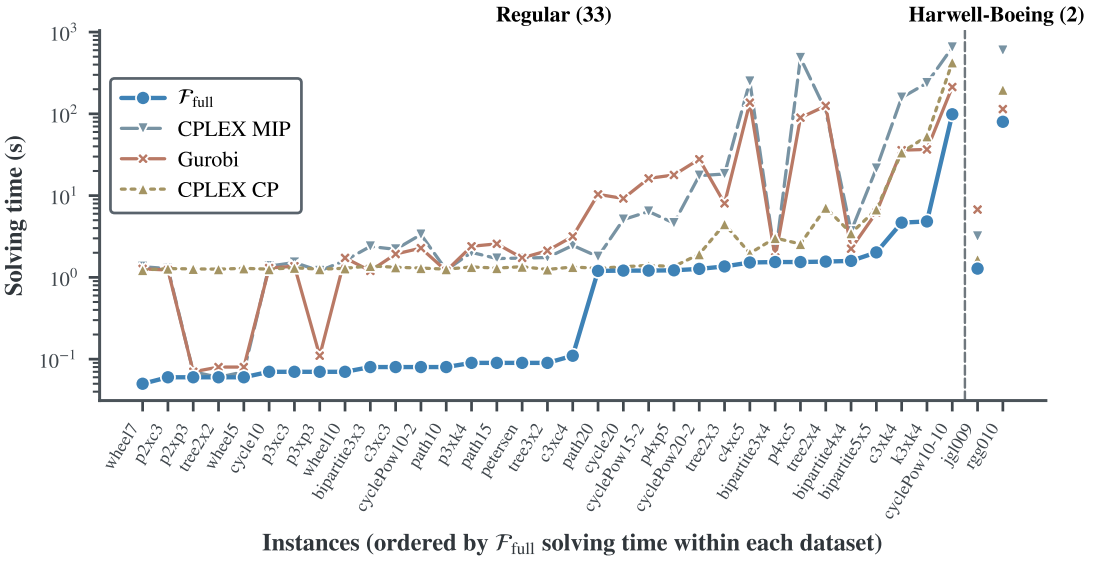


Figure 10: Solving-time comparison of $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP on the $n\times n$ host grid for the 35 instances certifying the same bandwidth.

Figure 9 shows the 53 instances on which the returned bandwidths or statuses differ. In the pairwise comparisons with Gurobi, CPLEX MIP, and CPLEX CP, $\mathcal{F}_{\text{full}}$ returns a lower bandwidth or returns a bandwidth when the compared solver returns none on 39, 43, and 9 instances, respectively. The reverse does not occur against Gurobi or CPLEX MIP and occurs only on `arc130` against CPLEX CP. These comparisons complement the certification counts by describing returned-bandwidth coverage and quality on the displayed subset.

Figure 10 restricts the solving-time comparison to the 35 instances on which all four methods certify the same bandwidth, comprising 33 Regular and 2 Harwell–Boeing instances. On this matched subset, $\mathcal{F}_{\text{full}}$ is faster than Gurobi and CPLEX CP on all 35 instances and faster than CPLEX MIP on 34, with one tie. The complete instance-level results for this host grid are reported in Table E.2.

5. Conclusion

This paper presented an exact SAT-based approach for the two-dimensional bandwidth minimization problem (2DBMP), together with an incremental threshold-search procedure for certifying optimal bandwidths. The approach is designed for the standard square-root-grid host and combines a compact SAT representation of vertex placement, occupancy, and Manhattan-distance constraints with symmetry breaking and incremental solving.

On the standard $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid, the proposed method certifies optimal bandwidths for 41 of 43 Regular instances and 42 of 93 Harwell–Boeing instances within the 3600 s time limit, substantially extending the optimality certification achieved by the previously published exact approach under a 72-hour time limit. It also certifies four bandwidth values that are lower than those reported by DRSA, the strongest heuristic method among the published approaches considered in this study; three of these improve upon all previously published comparison values and are certified optimal. Compared with the commercial exact solvers under the reported local experimental protocol, $\mathcal{F}_{\text{full}}$ generally achieves substantially broader optimality certification, particularly on the Harwell–Boeing benchmark, while maintaining competitive solving times on matched instances. On the standard host grid, it certifies 42 of 93 Harwell–Boeing instances, compared with 10, 8, and 20 for Gurobi, CPLEX MIP, and CPLEX CP, respectively. The additional experiments on $2\times\lceil n/2\rceil$ and $n\times n$ host grids further demonstrate that the approach remains effective under alternative host geometries.

The results should be interpreted primarily as evidence of the effectiveness of SAT for exact certification, rather than as evidence of dominance over heuristic methods, which may still provide better feasible incumbents when the exact solver reaches the time limit without completing a certificate. Moreover, the reported runtime comparisons are based on single runs under the specified experimental protocol, while published runtimes were obtained under different hardware

and experimental settings and are therefore not directly comparable or statistically validated. Overall, the proposed approach provides an effective exact method for the small- and medium-sized benchmark instances considered in this study, with fewer than 400 vertices, while larger and more challenging instances, particularly dense guest graphs, remain difficult for exact approaches. Future work will focus on stronger lower bounds, additional sound symmetry reductions, and more compact incremental encodings to improve scalability.

CRediT authorship contribution statement

Pham Quang Minh: Methodology, Software, Validation, Writing – review and editing. Dao Xuan Nghia: Methodology, Supervision, Validation, Writing – original draft, Writing – review and editing. To Van Khanh: Conceptualization, Investigation, Supervision, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The benchmark result files and source code used in this manuscript are available in the project repository: https://github.com/PQMlmaoxd/Two-Dimensional-Bandwidth-Minimization-Problem.

Appendix A ILP Linearization

This appendix records the linearization of the absolute-value constraint used in the reference ILP formulation.

Lemma 1 (Linearization of the absolute-value constraint)**.** *For a fixed placement and an edge $e = (u, v)$, let $x_u = \sum_{r=1}^{h}\sum_{c=1}^{h} r\, a_{urc}$, $y_u = \sum_{r=1}^{h}\sum_{c=1}^{h} c\, a_{urc}$, $x_v = \sum_{r=1}^{h}\sum_{c=1}^{h} r\, a_{vrc}$, and $y_v = \sum_{r=1}^{h}\sum_{c=1}^{h} c\, a_{vrc}$ denote the induced row and column coordinates. If the nonnegative split variables satisfy Constraints (5) and (6) for an edge $e = (u, v)$, then*

$$|x_u - x_v| + |y_u - y_v| \le d_e^{x,+} + d_e^{x,-} + d_e^{y,+} + d_e^{y,-}$$

Consequently, Constraint (7) implies that edge e has Manhattan distance at most β. Conversely, any fixed placement whose edge e has Manhattan distance at most β can be extended to nonnegative split variables satisfying Constraints (5)–(7).

Proof. Constraint (5) gives $x_u - x_v = d_e^{x,+} - d_e^{x,-}$. Since $d_e^{x,+}$ and $d_e^{x,-}$ are nonnegative, the triangle inequality gives

$$|x_u - x_v| = |d_e^{x,+} - d_e^{x,-}| \le d_e^{x,+} + d_e^{x,-}$$

The same argument applied to Constraint (6) gives

$$|y_u - y_v| \le d_e^{y,+} + d_e^{y,-}$$

Adding the two inequalities and using Constraint (7) proves the forward implication.

For the converse direction, set $d_e^{x,+} = \max\{x_u - x_v, 0\}$ and $d_e^{x,-} = \max\{x_v - x_u, 0\}$, and choose $d_e^{y,+}$ and $d_e^{y,-}$ analogously for the column difference. These values are nonnegative and at most $h - 1$ because all coordinates lie in $\{1, \dots, h\}$. They satisfy Constraints (5) and (6), and their total equals $|x_u - x_v| + |y_u - y_v|$. Therefore Constraint (7) holds whenever the edge has Manhattan distance at most β. □

Appendix B Correctness of the SAT Encoding

The first four lemmas establish the correctness of the SAT components, leading to Proposition 1 for the proposed full encoding. The remaining results establish the equivalence of the alternative configurations, the exactness of the incremental search, and the exactness of the encoding on the $2 \times \lceil n/2 \rceil$ host.

Lemma 2 (Position and occupancy correctness)**.** *Every satisfying assignment of either* $\mathcal{F}_{\text{pos}} \wedge \mathcal{F}_{\text{occ}}^{\text{seq}}$ *or* $\mathcal{F}_{\text{pos}} \wedge \mathcal{F}_{\text{occ}}^{\text{pair}}$ *determines an injective embedding* $\varphi : V \to H_h$ *through the position variables. Conversely, every injective embedding is represented by at least one satisfying assignment under each occupancy encoding. Thus, replacing* $\mathcal{F}_{\text{occ}}^{\text{seq}}$ *with* $\mathcal{F}_{\text{occ}}^{\text{pair}}$ *does not change the set of feasible embeddings.*

Proof. First, let an assignment satisfy either $\mathcal{F}_{\text{pos}} \wedge \mathcal{F}_{\text{occ}}^{\text{seq}}$ or $\mathcal{F}_{\text{pos}} \wedge \mathcal{F}_{\text{occ}}^{\text{pair}}$. Equations (12) and (13) ensure that each vertex v has a unique true row variable X_{v,r_v} and a unique true column variable Y_{v,c_v}. Define $\varphi(v) = (r_v, c_v)$.

Suppose first that the assignment satisfies $\mathcal{F}_{\text{occ}}^{\text{seq}}$. Equations (15)–(17) ensure that $A_{v,r,c}$ is true exactly when both $X_{v,r}$ and $Y_{v,c}$ are true. If two distinct vertices u and v were mapped to the same grid position (r, c), then both $A_{u,r,c}$ and $A_{v,r,c}$ would be true, contradicting the at-most-one constraint in Equation (18). Suppose instead that the assignment satisfies $\mathcal{F}_{\text{occ}}^{\text{pair}}$. If two distinct vertices u and v were mapped to the same position (r, c), the clause in Equation (14) indexed by u, v, and (r, c) would have all four literals false. This contradicts the assumption that $\mathcal{F}_{\text{occ}}^{\text{pair}}$ is satisfied. Thus, in both cases, no two vertices share a grid position, so φ is injective.

For the converse direction, let $\varphi : V \to H_h$ be an injective embedding. Set $X_{v,r}$ true exactly when r is the row of $\varphi(v)$, and set $Y_{v,c}$ true exactly when c is its column. These assignments satisfy the exactly-one requirements in Equations (12) and (13), and the auxiliary variables of their CNF encodings can be assigned accordingly.

For $\mathcal{F}_{\text{occ}}^{\text{seq}}$, set $A_{v,r,c}$ true exactly when $\varphi(v) = (r, c)$. The linking clauses in Equations (15)–(17) then hold. Injectivity ensures that at most one variable in $\{A_{v,r,c} : v \in V\}$ is true for each grid position, so the sequential-counter auxiliary variables can be assigned to satisfy Equation (18). For $\mathcal{F}_{\text{occ}}^{\text{pair}}$, injectivity ensures that no two distinct vertices occupy the same position. Hence, every clause in Equation (14) contains at least one true literal. Thus, the assignment can be extended to satisfy either conjunction. □

Lemma 3 (Distance-threshold semantics and cutoff validity)**.** *Suppose that the position variables satisfy* $\mathcal{F}_{\text{pos}}$*. For each edge* $e = (u, v) \in E$*, define* $a_x(e) = |x_u - x_v|$ *and* $a_y(e) = |y_u - y_v|$*. Every satisfying extension to* $\mathcal{F}_{\text{dist}}$ *satisfies* $T_{e,d}^q$ *if and only if* $d \le a_q(e)$ *for* $q \in \{x, y\}$ *and* $d \in \{1, \dots, h-1\}$*, and every position assignment has such an extension. A position assignment can be extended to satisfy* $\mathcal{F}_{\text{dist}}^{U^*}$ *if and only if* $a_q(e) \le U^*$ *for every edge* e *and axis* q*. In every such extension,* $T_{e,d}^q$ *is true if and only if* $d \le a_q(e)$ *for* $d \in \{1, \dots, \min\{U^*, h-1\}\}$*. Consequently, the cutoff-aware encoding preserves every embedding whose bandwidth is at most* U^**.*

Proof. Fix an edge $e = (u, v)$ and an axis $q \in \{x, y\}$. The position component selects unique coordinates for u and v on this axis, so their separation $a = a_q(e)$ is fixed. If $a = 0$ and threshold 1 is represented, Equation (23) forces $T_{e,1}^q$ to be false. Equation (24) then prevents any higher represented threshold from being true, because a true $T_{e,d}^q$ would propagate down to $T_{e,1}^q$. If no threshold is represented, the threshold-semantics statement is vacuous.

Now suppose that $a > 0$. According to the order of the two selected coordinates, Equation (19) or Equation (20) forces $T_{e,a}^q$ to be true whenever that threshold is represented. Repeated application of Equation (24) therefore forces every $T_{e,d}^q$ with $d \le a$ to be true. If $a + 1$ is represented, Equation (21) or Equation (22) forces $T_{e,a+1}^q$ to be false. Any true threshold above $a + 1$ would propagate down to $T_{e,a+1}^q$ and produce a contradiction, while no threshold above a exists when a is the largest represented value. Thus, $T_{e,d}^q$ is true exactly for the represented thresholds satisfying $d \le a$.

For the reverse direction, set each represented $T_{e,d}^q$ true exactly when $d \le a_q(e)$. The activation, deactivation, zero-separation, and monotonicity clauses in Equations (19)–(24) then hold by construction. In the cutoff-aware encoding, Equation (27) is satisfied exactly when no edge has an axis separation greater than U^*. If an embedding has bandwidth at most U^*, then $a_x(e) + a_y(e) \le U^*$ for every edge e. Since both axis separations are nonnegative, $a_x(e) \le U^*$ and $a_y(e) \le U^*$. Equation (27) is therefore satisfied, and the threshold assignment above gives a satisfying extension to $\mathcal{F}_{\text{dist}}^{U^*}$. □

Lemma 4 (Bandwidth-component correctness)**.** *Let the position variables satisfy* $\mathcal{F}_{\text{pos}}$*. For every integer* K *with* $0 \le K \le U^*$*, the position assignment can be extended to satisfy* $\mathcal{F}_{\text{dist}}^{U^*}$ *together with the form of* Δ_K *in Equation* (30a) *if and only if every guest edge has Manhattan distance at most* K*. For every integer* K *with* $0 \le K \le 2(h-1)$*, the same equivalence holds for* $\mathcal{F}_{\text{dist}}$ *together with the form of* Δ_K *in Equation* (30b)*.*

Proof. Consider either distance encoding and let $L = \min\{U^*, h-1\}$ in the cutoff-aware case and $L = h - 1$ in the full-distance case. Fix an edge $e = (u, v)$ and define $a_x(e) = |x_u - x_v|$ and $a_y(e) = |y_u - y_v|$.

Suppose first that the distance encoding and the applicable form of Δ_K are satisfied. Lemma 3 gives $0 \le a_x(e), a_y(e) \le L$ and makes $T^x_{e,d}$ and $T^y_{e,d}$ true exactly up to the corresponding axis separations. If $a_x(e) \ge K+1$, then $K+1 \le a_x(e) \le L$, so the unit clause $\neg T^x_{e,K+1}$ is present and is violated. The same argument on the other axis shows that $a_x(e) \le K$ and $a_y(e) \le K$. For $K = 0$, these two inequalities give $a_x(e) + a_y(e) \le K$. For $K \ge 1$, suppose that $a_x(e) + a_y(e) \ge K+1$. Both axis separations are then positive. Setting $i = a_x(e)$ gives $1 \le i \le K$, $i \le L$, and $K - i + 1 \le a_y(e) \le L$. The corresponding binary clause in Equation (30a) or Equation (30b) is violated because both threshold variables are true. Hence every edge satisfies $a_x(e) + a_y(e) \le K$.

For the other direction, suppose that every edge satisfies $a_x(e) + a_y(e) \le K$. In the cutoff-aware case, $a_x(e), a_y(e) \le K \le U^*$, and the grid-coordinate domains also give $a_x(e), a_y(e) \le h-1$. Hence, $a_x(e), a_y(e) \le \min\{U^*, h-1\}$. In the full-distance case, the grid-coordinate domains give $a_x(e), a_y(e) \le h-1$. Lemma 3 therefore provides the required threshold valuation for either distance encoding. Any represented unit clause at threshold $K+1$ is satisfied because neither axis separation exceeds K. If both threshold variables in a binary clause were true, then $a_x(e) \ge i$ and $a_y(e) \ge K - i + 1$, which would imply $a_x(e) + a_y(e) \ge K+1$. Therefore every binary clause is also satisfied, and the conclusion holds for both forms of Δ_K. □

Lemma 5 (Q1 symmetry preservation)**.** *Every injective embedding has an embedding obtained by horizontal and vertical reflections whose anchor vertex lies in Q_1. These reflections preserve injectivity and the Manhattan distance of every guest edge, so adding $\mathcal{F}_{\text{sym}}$ does not change bandwidth-K feasibility.*

Proof. Let φ be an injective embedding and write $\varphi(a) = (r_a, c_a)$ for the position of the selected anchor vertex. If $r_a > \lceil h/2 \rceil$, reflect every grid position horizontally by replacing its row r with $h+1-r$. The new anchor row satisfies $h+1-r_a \le \lfloor h/2 \rfloor \le \lceil h/2 \rceil$. If $c_a > \lceil h/2 \rceil$, apply the analogous vertical reflection by replacing every column c with $h+1-c$, which places the anchor column in the same allowed range.

Each reflection is a bijection of H_h, so applying it to all vertices preserves injectivity. A horizontal reflection preserves row differences because $|(h+1-r_u) - (h+1-r_v)| = |r_u - r_v|$, and a vertical reflection preserves column differences by the same argument. Their composition therefore preserves the Manhattan distance of every guest edge and leaves the bandwidth unchanged. The transformed anchor lies in Q_1, so its position variables satisfy Equations (28) and (29), establishing the claimed preservation. □

Proposition 1 (Correctness of $\mathcal{F}_{\text{full}}(K)$)**.** *For every integer K with $0 \le K \le U^*$, the formula $\mathcal{F}_{\text{full}}(K)$ is satisfiable if and only if there exists an injective embedding $\varphi : V \to H_h$ such that $\beta(G, \varphi) \le K$.*

Proof. Suppose that $\mathcal{F}_{\text{full}}(K)$ is satisfiable. By Lemma 2, its position and sequential occupancy components determine an injective embedding $\varphi : V \to H_h$. Lemma 4, applied to $\mathcal{F}^{U^*}_{\text{dist}}$ and the form of Δ_K in Equation (30a), shows that every guest edge has Manhattan distance at most K. It follows that $\beta(G, \varphi) \le K$.

For the reverse direction, let φ be an injective embedding with $\beta(G, \varphi) \le K$. Lemma 5 provides an injective embedding φ' with the same bandwidth whose anchor lies in Q_1. Lemma 2 extends φ' to a satisfying assignment of $\mathcal{F}_{\text{pos}} \wedge \mathcal{F}^{\text{seq}}_{\text{occ}}$, while Lemmas 3 and 4 extend it to satisfy $\mathcal{F}^{U^*}_{\text{dist}} \wedge \Delta_K$. Since the auxiliary variables introduced by the occupancy and distance components are disjoint, these extensions can be combined. The anchor position satisfies $\mathcal{F}_{\text{sym}}$, so all components in Equation (11) are satisfied. □

Lemma 6 (Equivalence of the alternative configurations)**.** *For every K with $0 \le K \le U^*$, each alternative encoding configuration in Table 1 is satisfiable if and only if $\mathcal{F}_{\text{full}}(K)$ is satisfiable.*

Proof. Any satisfying assignment of $\mathcal{F}_{\text{full}}(K)$ satisfies $\mathcal{F}_{\text{no-sym}}(K)$ after the symmetry clauses are omitted. In the other direction, Lemmas 2 and 4 show that a model of $\mathcal{F}_{\text{no-sym}}(K)$ determines an injective embedding of bandwidth at most K. Proposition 1 then shows that $\mathcal{F}_{\text{full}}(K)$ is satisfiable.

For $\mathcal{F}_{\text{pair}}(K)$, Lemma 2 shows that pairwise and sequential occupancy represent the same injective embeddings. A model of $\mathcal{F}_{\text{pair}}(K)$ therefore yields an embedding of bandwidth at most K by Lemma 4, and Proposition 1 gives a model of $\mathcal{F}_{\text{full}}(K)$. Conversely, an embedding supplied by Proposition 1 can be reflected into Q_1 by Lemma 5, represented with pairwise occupancy by Lemma 2, and extended to the cutoff-aware distance and bandwidth components by Lemmas 3 and 4.

For $\mathcal{F}_{\text{full-dist}}(K)$, a satisfying assignment determines an injective embedding by Lemma 2, and Lemma 4 shows that its bandwidth is at most K. Proposition 1 then gives a satisfying assignment of $\mathcal{F}_{\text{full}}(K)$. Conversely, Proposition 1 supplies an injective bandwidth-K embedding from a model of $\mathcal{F}_{\text{full}}(K)$. Lemma 5 places its anchor in Q_1, Lemma 2

represents its position and occupancy components, and Lemmas 3 and 4 extend it to satisfy the full-distance component and the applicable form of Δ_K. This completes the equivalence of all four configurations. □

Lemma 7 (Threshold monotonicity and cumulative threshold clauses)**.** *Let $\mathcal{F}_{\mathrm{cfg}}(K)$ denote an encoding configuration in Table 1. For configurations containing $\mathcal{F}_{\mathrm{dist}}^{U^*}$, satisfiability is monotone over $0 \le K \le U^*$. For $\mathcal{F}_{\text{full-dist}}(K)$, satisfiability is monotone over $0 \le K \le 2(h-1)$. Moreover, let $\mathcal{B}_{\mathrm{cfg}}$ denote the conjunction of the K-independent components of the selected configuration. For any decreasing sequence $U^* \ge K_1 > \cdots > K_t \ge 1$ in a cutoff-aware configuration, or $2(h-1) \ge K_1 > \cdots > K_t \ge 1$ in $\mathcal{F}_{\text{full-dist}}(K)$, the cumulative formula $\mathcal{B}_{\mathrm{cfg}} \wedge \bigwedge_{j=1}^{t} \Delta_{K_j}$ is satisfiable if and only if $\mathcal{F}_{\mathrm{cfg}}(K_t)$ is satisfiable.*

Proof. For configurations containing $\mathcal{F}_{\mathrm{dist}}^{U^*}$, Proposition 1 and Lemma 6 show that $\mathcal{F}_{\mathrm{cfg}}(K)$ is satisfiable exactly when an injective embedding of bandwidth at most K exists over $0 \le K \le U^*$. For $\mathcal{F}_{\text{full-dist}}(K)$, the same characterization over $0 \le K \le 2(h-1)$ follows from Lemmas 2, 3, 4, and 5. In either case, the same embedding has bandwidth at most every larger threshold in the applicable range, so the corresponding configuration is satisfiable at each such threshold. This establishes threshold monotonicity for every configuration in Table 1.

The cumulative formula contains $\mathcal{B}_{\mathrm{cfg}} \wedge \Delta_{K_t}$, so any model of the cumulative formula is also a model of $\mathcal{F}_{\mathrm{cfg}}(K_t)$. For the reverse direction, let an assignment satisfy $\mathcal{F}_{\mathrm{cfg}}(K_t)$. Its position and threshold variables describe an embedding of bandwidth at most K_t by Lemmas 3 and 4. Since $K_j > K_t$ for every earlier threshold, the same threshold valuation satisfies each Δ_{K_j} by Lemma 4. The assignment therefore satisfies the complete cumulative formula.

Any learned clause retained by a sound SAT solver is a logical consequence of the clauses present when it was learned. Every later cumulative formula contains those earlier clauses, so retaining learned clauses also preserves the satisfiability equivalence at the current threshold. Therefore, keeping both earlier threshold clauses and learned clauses changes the solver state but not the bandwidth decision problem being solved. □

Proposition 2 (Exactness of the incremental search)**.** *Assume that G contains at least one edge whose endpoints are distinct, U^* is a valid upper bound, and the incremental procedure in Table 2 uses an encoding configuration from Table 1. If no timeout occurs, the procedure terminates with status* OPTIMAL *and returns the certified optimal value $\beta^*(G)$. If a timeout occurs, any returned incumbent is only a feasible upper bound.*

Proof. Because U^* is a valid upper bound, an injective embedding of bandwidth at most U^* exists. For a configuration containing $\mathcal{F}_{\mathrm{dist}}^{U^*}$, Proposition 1 and Lemma 6 show that the selected configuration is satisfiable at its initial threshold U^*. For $\mathcal{F}_{\text{full-dist}}(K)$, every embedding on the $h \times h$ grid has bandwidth at most $2(h-1)$, and Lemmas 2, 3, 4, and 5 show that this configuration is satisfiable at its initial threshold $2(h-1)$. Hence, unless the first solver call times out, the procedure obtains a feasible incumbent before any unsatisfiable result can occur.

Consider a satisfiable call at threshold K. The decoded model gives an embedding with actual bandwidth $B \le K$, so B is a feasible upper bound and $\beta^*(G) \le B$. The next threshold is $B-1$, which is strictly smaller than K. If $B > \beta^*(G)$, integrality gives $B-1 \ge \beta^*(G)$, and the next decision problem remains satisfiable. If $B = \beta^*(G) > 1$, the next threshold is below the optimal value and is unsatisfiable. Lemma 7 ensures that the cumulative solver state has exactly the same satisfiability status as the selected configuration at this new threshold.

The tested thresholds decrease after every satisfiable call, so the process is finite when no timeout occurs. It either reaches an incumbent $B = \beta^*(G) > 1$ followed by an unsatisfiable call at $B-1$, or reaches a feasible incumbent $B = 1$. In the first case, feasibility at B and infeasibility at $B-1$ certify that B is the optimal value. In the second case, injectivity and the existence of an edge with distinct endpoints imply $\beta^*(G) \ge 1$, so the feasible value 1 is also certified optimal. The smart jump omits only thresholds from B through $K-1$, all of which admit the current embedding. Every embedding with bandwidth smaller than B remains feasible at the next tested threshold $B-1$, so the jump excludes no improving embedding.

A timeout establishes neither satisfiability nor unsatisfiability at the current threshold. Any incumbent obtained earlier remains a feasible upper bound, but the procedure has not excluded a smaller bandwidth. □

Lemma 8 (Exactness on the $2 \times \lceil n/2 \rceil$ host)**.** *Let U be a feasible initial threshold for $\mathcal{F}_{\mathrm{full}}(K)$ instantiated on the $2 \times \lceil n/2 \rceil$ host. For every integer K with $0 \le K \le U$, $\mathcal{F}_{\mathrm{full}}(K)$ on the $2 \times \lceil n/2 \rceil$ host is satisfiable if and only if there exists an injective embedding on the $2 \times \lceil n/2 \rceil$ host with bandwidth at most K. Consequently, a feasible bandwidth B is a certified optimal value if $B = 1$ or if the incremental solver returns* UNSAT *at $B-1$. If a solver call times out before either condition is established, any returned incumbent is only a feasible upper bound.*

Proof. On the $2\times\lceil n/2\rceil$ host, the row-coordinate domain is $\{1,2\}$ and the column-coordinate domain is $\{1,\dots,\lceil n/2\rceil\}$. The position and occupancy argument in Lemma 2 remains valid for these two coordinate domains and therefore represents exactly the injective embeddings on the $2\times\lceil n/2\rceil$ host.

For an embedding φ, write $\varphi(v)=(r_v,c_v)$. For every guest edge (u,v), the row separation $|r_u-r_v|$ is at most 1, while the column separation $|c_u-c_v|$ is at most $\lceil n/2\rceil-1$. The row-distance thresholds range up to $\min\{U,1\}$, and the column-distance thresholds range up to $\min\{U,\lceil n/2\rceil-1\}$. Applying the argument in Lemma 3 separately to these two threshold ranges shows that the corresponding threshold variables represent the row and column separations exactly. The argument in Lemma 4 then shows that the distance components and Δ_K are satisfiable if and only if

$$|r_u-r_v|+|c_u-c_v|\le K$$

for every guest edge. The left-hand side is the Manhattan distance between the positions of u and v.

A row reflection maps r to $3-r$, while a column reflection maps c to $\lceil n/2\rceil+1-c$. Both reflections are bijections of the $2\times\lceil n/2\rceil$ host and preserve the row and column separations. The symmetry-preservation argument in Lemma 5 therefore remains valid. Adding $\mathcal{F}_{\mathrm{sym}}$ does not change bandwidth-K feasibility. This proves the stated satisfiability equivalence.

The argument in Lemma 7 applies with the row and column threshold ranges above, so the cumulative solver state at each tested threshold has the same satisfiability status as $\mathcal{F}_{\mathrm{full}}(K)$ on the $2\times\lceil n/2\rceil$ host. The certification and timeout arguments in Proposition 2 therefore apply unchanged. □

Appendix C Initial Upper Bound for the $2\times\lceil n/2\rceil$ Grid

This section describes how a valid initial upper bound is obtained for the $2\times\lceil n/2\rceil$ grid. Throughout this section, $m=\lceil n/2\rceil$ and $H_{2,m}=[2]\times[m]$. Because $\delta(n)$ concerns unrestricted grid positions rather than positions restricted to $H_{2,m}$, it supplies only the starting scale for the candidate thresholds. The initialization then combines these candidates with an instance-specific lower bound and SAT feasibility checks, and adopts a candidate as the initial upper bound only after SAT verifies a feasible embedding at that threshold.

The lower bound combines a maximum-degree capacity condition with a connected-component condition. For an integer bandwidth threshold $B\ge 1$, Equation (C.1) presents the strip-capacity term $S_m(B)$:

$$S_m(B)=\big(\min\{2B+1,m\}-1\big)+\min\{2B-1,m\}. \tag{C.1}$$

For any position $p\in H_{2,m}$, the first summand in Equation (C.1) bounds the number of other positions in the same row as p that lie within Manhattan distance B of p. The second summand in Equation (C.1) bounds the corresponding number of positions in the other row. Here, $\Delta(G)$ denotes the maximum degree of G, and $D_C=\mathrm{diam}_G(C)$ for each connected component C with $|C|\ge 2$. The capacity requirement $\Delta(G)\le S_m(B)$ leads to the maximum-degree bound, while comparing the host distance forced by $|C|$ distinct positions with D_C leads to the component bound. Equations (C.2) and (C.3) present these bounds, respectively. Equation (C.4) takes their maximum, and Lemma 9 establishes its validity:

$$\mathrm{LB}_{\text{strip-degree}}(G)=\min\left\{B\in\mathbb{Z}_{\ge 1}\,:\,\Delta(G)\le S_m(B)\right\}, \tag{C.2}$$

$$\mathrm{LB}_{\text{component}}(G)=\max_{\substack{C\text{ connected component of }G\\ |C|\ge 2}}\left\lceil\frac{\lceil |C|/2\rceil}{D_C}\right\rceil, \tag{C.3}$$

$$\mathrm{LB}_{2,m}(G)=\max\left\{\mathrm{LB}_{\text{strip-degree}}(G),\mathrm{LB}_{\text{component}}(G)\right\}. \tag{C.4}$$

The lower bound determines where feasibility testing begins, whereas SAT determines whether a selected candidate is feasible. Equation (C.5) presents the candidate-threshold sequence generated from $\delta(n)$:

$$P_j=\min\left\{m,\left\lceil\left(1+\frac{j}{2}\right)\delta(n)\right\rceil\right\},\qquad j=0,1,2,\dots. \tag{C.5}$$

The sequence starts with $\delta(n)$, increases its multiplier in steps of 0.5, and caps each candidate at the host diameter m. The initialization selects the first candidate P_j satisfying $P_j\ge\mathrm{LB}_{2,m}(G)$ and tests it with the cutoff-aware SAT encoding. If the result is UNSAT, the threshold is increased one unit at a time and the encoding is rebuilt until SAT

verifies a feasible embedding. The first satisfiable threshold is a valid upper bound and is used as both the distance cutoff and the initial threshold of the descending search. By Lemma 9, candidate values below $\mathrm{LB}_{2,m}(G)$ are infeasible and need not be tested during initialization. In addition, this lower bound can be used to certify optimality when it equals a verified feasible bandwidth.

Lemma 9 (Lower bound for $H_{2,m}$)**.** *Assume that $E \neq \emptyset$. Every injective embedding $\varphi : V \to H_{2,m}$ satisfies:*

$$\beta(G, \varphi) \geq \mathrm{LB}_{2,m}(G). \tag{C.6}$$

Proof. Fix an injective embedding $\varphi : V \to H_{2,m}$ and write $B = \beta(G, \varphi)$. Since $E \neq \emptyset$, $B \geq 1$. Choose a vertex $v^\star$ of maximum degree. At most $\min\{2B + 1, m\} - 1$ positions other than $\varphi(v^\star)$ in its row can be within Manhattan distance B. In the other row, the row separation is one, so at most $\min\{2B - 1, m\}$ positions can be within distance B. Injectivity places the distinct neighbors of $v^\star$ in these positions, which gives $\Delta(G) \leq S_m(B)$. Hence,

$$B \geq \mathrm{LB}_{\text{strip-degree}}(G). \tag{C.7}$$

For the component bound, fix a connected component C with $|C| \geq 2$, and let $q = \lceil |C|/2 \rceil$. Any $|C|$ distinct positions in $H_{2,m}$ contain two positions at Manhattan distance at least q. To see this, let a and b be the minimum and maximum occupied columns and set $L = b - a$. Since each column contains at most two positions, $L \geq q - 1$. If $L \geq q$, the claim follows from the column separation. If $L = q - 1$, the number of occupied positions forces an opposite-row pair across the two extreme columns, whose Manhattan distance is q.

Let $u, v \in C$ occupy such a pair of positions. A shortest path between u and v in G has length at most D_C, and every edge on this path has host distance at most B. The triangle inequality gives

$$q \leq d_{H_{2,m}}\big(\varphi(u), \varphi(v)\big) \leq B D_C. \tag{C.8}$$

Therefore,

$$B \geq \left\lceil \frac{q}{D_C} \right\rceil = \left\lceil \frac{\lceil |C|/2 \rceil}{D_C} \right\rceil. \tag{C.9}$$

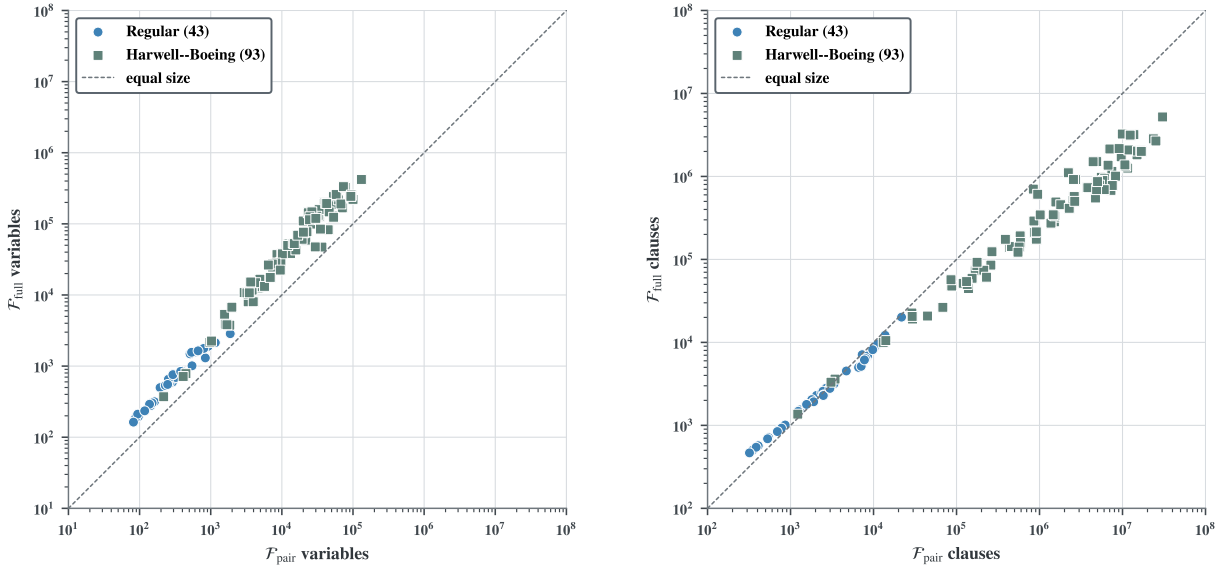
Taking the maximum over all non-singleton connected components gives $B \geq \mathrm{LB}_{\text{component}}(G)$. Combining the two inequalities proves the claim. □

Appendix D Encoding-Size Comparison in the Ablation Study

Table D.1
Variable and clause counts for the SAT variants on the Regular and Harwell–Boeing sets using the $\lceil \sqrt{n} \rceil \times \lceil \sqrt{n} \rceil$ host grid. The counts exclude Δ_K.

Dataset	Variant	Median variables	Max variables	Median clauses	Max clauses
Regular	$\mathcal{F}_{\text{full}}$	716	2,855	2,494	20,150
Regular	$\mathcal{F}_{\text{no-sym}}$	716	2,855	2,648	20,860
Regular	$\mathcal{F}_{\text{pair}}$	312	1,880	2,508	21,754
Regular	$\mathcal{F}_{\text{full-dist}}$	716	2,855	2,494	20,150
Regular	non-incremental $\mathcal{F}_{\text{full}}$	716	2,855	2,494	20,150
Harwell–Boeing	$\mathcal{F}_{\text{full}}$	69,263	420,492	512,972	5,210,530
Harwell–Boeing	$\mathcal{F}_{\text{no-sym}}$	69,263	420,492	526,444	5,225,972
Harwell–Boeing	$\mathcal{F}_{\text{pair}}$	20,340	131,292	2,225,780	30,372,830
Harwell–Boeing	$\mathcal{F}_{\text{full-dist}}$	69,263	420,492	512,972	5,210,530
Harwell–Boeing	non-incremental $\mathcal{F}_{\text{full}}$	69,263	420,492	512,972	5,210,530

Table D.1 reports the median and maximum variable and clause counts after the fixed encoding components are constructed and before Δ_K is added. $\mathcal{F}_{\text{pair}}$ has lower median and maximum variable counts but the largest maximum



(a) Variables (b) Clauses

Figure D.1: Comparison of variable and clause counts between $\mathcal{F}_{\text{full}}$ and $\mathcal{F}_{\text{pair}}$ for the Regular and Harwell–Boeing sets on the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid. The counts exclude Δ_K. Points below the diagonal indicate smaller counts for $\mathcal{F}_{\text{full}}$.

clause count in both datasets. For every instance in both datasets, $U^* \geq \lceil\sqrt{n}\rceil - 1$, where $\lceil\sqrt{n}\rceil - 1$ is the largest possible separation along one axis of the host grid. The cutoff therefore removes no distance thresholds, and the index set in Equation (27) is empty. Consequently, no cutoff clauses are generated, and $\mathcal{F}_{\text{full}}$ and $\mathcal{F}_{\text{full-dist}}$ have identical variable and clause counts. The non-incremental variant also matches $\mathcal{F}_{\text{full}}$ because the two variants use the same encoded components before Δ_K is added.

Figure D.1 gives an instance-level view of the occupancy-encoding tradeoff between $\mathcal{F}_{\text{full}}$ and $\mathcal{F}_{\text{pair}}$. The variable panel shows that $\mathcal{F}_{\text{full}}$ uses more variables on all displayed instances. In contrast, the clause panel shows that $\mathcal{F}_{\text{pair}}$ incurs substantially larger clause counts on many larger Harwell–Boeing instances, while some smaller instances remain above the diagonal.

Appendix E Detailed Benchmark Tables

The detailed results used in Section 4 are reported in Tables E.1 and E.2. The locally evaluated methods are $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP, with solving times reported in seconds. Boldface marks the three certified bandwidths in Table E.1 that improve every published comparison value considered in this study. Within each host-grid setting, italics mark the best results, ranked first by lower bandwidth, then by certification status, and finally by lower solving time when the reported times are directly comparable. In Table E.2, dashes in the $n \times n$ block mark instances that could not be evaluated under the available hardware resources.

Table E.1: Detailed comparison of published bandwidths from M3, BVNS, IG, RVNS, DRSA, and RLTS and locally evaluated results from $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP on the $\lceil\sqrt{n}\rceil \times \lceil\sqrt{n}\rceil$ host grid for Regular and Harwell–Boeing instances.

Instance	n	$\lvert E\rvert$	M3	BVNS	IG	RVNS	DRSA	RLTS	$\mathcal{F}_{\text{full}}$	Gurobi	CPLEX MIP	CPLEX CP
Regular												
`wheel5`	5	8	*2**	2	2	–	–	2	*2* (0.03)*	2* (0.07)	2* (1.23)	2* (1.19)
`bipartite3x3`	6	9	*2**	2	2	–	–	2	*2* (0.04)*	2* (0.07)	2* (1.19)	2* (1.20)
`p2xc3`	6	9	*2**	2	2	–	–	2	*2* (0.03)*	2* (0.07)	2* (1.21)	2* (1.20)
`p2xp3`	6	7	*1**	2	1	–	–	1	*1* (0.04)*	1* (0.06)	1* (1.25)	1* (1.19)
`bipartite3x4`	7	12	*3**	3	3	–	–	3	*3* (0.05)*	3* (1.17)	3* (0.09)	3* (1.22)
`tree2x2`	7	6	*1**	1	1	–	–	1	*1* (0.04)*	1* (0.07)	1* (1.20)	1* (1.20)
`wheel7`	7	12	*2**	2	2	–	–	2	*2* (0.04)*	2* (0.07)	2* (1.21)	2* (1.21)

Table E.1: Detailed comparison on the $\lceil\sqrt{n}\rceil\times\lceil\sqrt{n}\rceil$ host grid for Regular and Harwell–Boeing instances (continued).

Instance	n	$\lvert E\rvert$	M3	BVNS	IG	RVNS	DRSA	RLTS	$\mathcal{F}_{\text{full}}$	Gurobi	CPLEX MIP	CPLEX CP
bipartite4x4	8	16	*3**	3	3	–	–	3	*3* (0.05)*	3* (1.20)	3* (1.19)	3* (1.24)
c3xc3	9	18	*2**	2	2	–	–	2	*2* (0.04)*	2* (0.08)	2* (0.07)	2* (1.21)
p3xc3	9	15	*2**	2	2	–	–	2	*2* (0.03)*	2* (0.08)	2* (1.25)	2* (1.22)
p3xp3	9	12	*1**	2	1	–	–	1	*1* (0.03)*	1* (0.07)	1* (1.21)	1* (1.20)
bipartite5x5	10	25	*3**	3	3	–	–	3	*3* (0.07)*	3* (1.31)	3* (1.27)	3* (1.29)
cycle10	10	10	*1**	1	1	–	–	1	*1* (0.04)*	1* (0.07)	1* (0.07)	1* (1.20)
cyclePow10-10	10	45	*4**	4	4	–	–	4	*4* (2.37)*	4* (2.67)	4* (3.57)	4* (8.84)
cyclePow10-2	10	20	*2**	2	2	–	–	2	*2* (0.04)*	2* (1.17)	2* (0.08)	2* (1.20)
path10	10	9	*1**	1	1	–	–	1	*1* (0.04)*	1* (0.07)	1* (1.24)	1* (1.21)
petersen	10	15	*2**	2	2	–	–	2	*2* (0.04)*	2* (1.22)	2* (1.50)	2* (1.22)
wheel10	10	18	*2**	2	2	–	–	2	*2* (0.05)*	2* (0.09)	2* (1.20)	2* (1.21)
c3xc4	12	24	*2**	2	2	–	–	2	*2* (0.04)*	2* (1.26)	2* (0.11)	2* (1.21)
c3xk4	12	30	*3**	3	3	–	–	3	*3* (0.10)*	3* (2.02)	3* (1.36)	3* (1.37)
k3xk4	12	30	*3**	3	3	–	–	3	*3* (1.16)*	3* (1.98)	3* (1.32)	3* (1.38)
p3xk4	12	26	*2**	2	2	–	–	2	*2* (0.04)*	2* (1.53)	2* (1.58)	2* (1.22)
tree2x3	13	12	*2**	2	2	–	–	2	*2* (0.04)*	2* (1.22)	2* (1.42)	2* (1.22)
bipartite7x8	15	56	*4**	4	4	–	–	4	4* (22.01)	4* (3.01)	*4* (1.64)*	4* (450.14)
cycle15	15	15	*2**	2	2	–	–	2	*2* (1.17)*	2* (1.67)	2* (1.34)	2* (1.30)
cyclePow15-10	15	105	–	6	6	–	–	6	6* (108.85)	6* (640.33)	6* (27.41)	*6* (10.67)*
cyclePow15-2	15	30	*2**	2	2	–	–	2	*2* (0.04)*	2* (1.38)	2* (0.11)	2* (1.23)
path15	15	14	*1**	2	1	–	–	1	*1* (0.04)*	1* (0.10)	1* (0.09)	1* (1.23)
tree3x2	15	14	*2**	2	2	–	–	2	*2* (0.04)*	2* (1.34)	2* (1.51)	2* (1.23)
wheel15	15	28	*3**	3	3	–	–	3	3* (1.48)	*3* (0.10)*	3* (1.62)	3* (4.33)
bipartite10x10	20	100	–	5	5	–	–	5	5 (FEAS)	5* (45.15)	5* (92.25)	*5* (20.18)*
c4xc5	20	40	*2**	3	2	–	–	2	*2* (0.06)*	2* (4.99)	2* (1.66)	2* (1.28)
c4xk5	20	60	*3**	3	3	–	–	3	*3* (1.26)*	3* (6.77)	3* (77.34)	3* (4.64)
cycle20	20	20	*1**	1	1	–	–	1	*1* (0.04)*	1* (1.23)	1* (1.21)	1* (1.21)
cyclePow20-10	20	190	–	*6*	*6*	–	–	*6*	*6 (FEAS)*	*6 (FEAS)*	*6 (FEAS)*	*6 (FEAS)*
cyclePow20-2	20	40	*2**	2	2	–	–	2	*2* (0.05)*	2* (1.74)	2* (1.62)	2* (1.27)
k4xk5	20	70	*4**	4	4	–	–	4	*4* (2312.08)*	4 (FEAS)	4 (FEAS)	4* (2754.32)
p4xc5	20	35	*2**	3	2	–	–	2	*2* (0.06)*	2* (2.50)	2* (2.04)	2* (1.25)
p4xk5	20	55	*3**	3	3	–	–	3	*3* (1.35)*	3* (14.70)	3* (28.16)	3* (2.49)
p4xp5	20	31	*1**	2	1	–	–	1	*1* (0.05)*	1* (2.55)	1* (1.23)	1* (1.22)
path20	20	19	*1**	1	1	–	–	1	*1* (0.05)*	1* (1.26)	1* (1.22)	1* (1.22)
wheel20	20	38	–	3	3	–	–	3	3* (5.97)	3* (2.69)	*3* (1.53)*	3* (71.87)
tree2x4	21	20	*2**	2	2	–	–	2	*2* (0.06)*	2* (1.44)	2* (1.32)	2* (1.27)
Harwell–Boeing												
jgl009	9	50	–	–	3	–	3	3	*3* (0.04)*	3* (1.65)	3* (0.07)	3* (1.25)
rgg010	10	76	–	–	–	–	4	–	*4* (2.33)*	4* (2.37)	4* (3.58)	4* (8.53)
jgl011	11	76	–	–	4	–	4	4	*4* (2.02)*	4* (2.41)	4* (4.03)	4* (8.12)
can___24	24	92	–	–	3	–	3	3	*3* (1.36)*	3* (14.83)	3* (2.77)	3* (2.66)
lap_25	25	97	–	–	–	–	2	–	*2* (1.17)*	2* (1.37)	2* (1.78)	2* (1.27)
fidap005	27	279	–	–	–	–	4	–	*4* (15.84)*	4* (553.90)	4* (675.42)	4* (52.33)
pores_1	30	180	–	–	3	–	3	3	*3* (2.06)*	3* (18.25)	3* (491.58)	3* (3.63)
ibm32	32	126	–	–	*4*	–	*4*	*4*	*4 (FEAS)*	*4 (FEAS)*	*4 (FEAS)*	*4 (FEAS)*
bcspwr01	39	85	–	–	2	–	2	2	*2* (1.17)*	2* (13.83)	2 (FEAS)	2* (1.36)
fidapm05	42	520	–	–	–	–	4	–	*4* (109.25)*	4 (FEAS)	5 (FEAS)	4* (710.15)
bcsstk01	48	224	6	5	*4*	5	*4*	*4*	*4 (FEAS)*	5 (FEAS)	6 (FEAS)	5 (FEAS)
bcspwr02	49	108	–	–	2	–	2	2	*2* (2.14)*	2* (39.37)	3 (FEAS)	2* (3.78)
curtis54	54	291	–	–	3	–	3	3	*3* (3.15)*	3 (FEAS)	4 (FEAS)	3* (20.18)
will57	57	281	–	–	3	–	3	3	*3* (3.23)*	3 (FEAS)	4 (FEAS)	3* (83.75)
dwt___59	59	163	–	–	3	–	2	2	2* (2.87)	3 (FEAS)	4 (FEAS)	*2* (2.07)*
impcol_b	59	312	–	–	*5*	–	*5*	*5*	*5 (FEAS)*	*5 (FEAS)*	6 (FEAS)	*5 (FEAS)*
can___61	61	309	–	–	*4*	–	*4*	*4*	*4 (FEAS)*	*4 (FEAS)*	6 (FEAS)	*4 (FEAS)*
bfw62a	62	450	–	–	–	–	4	–	***3* (584.02)***	4 (FEAS)	5 (FEAS)	4 (FEAS)
bfw62b	62	342	–	–	–	–	3	–	*3* (8.77)*	3* (75.08)	4 (FEAS)	3* (44.22)
can___62	62	140	*2**	2	2	3	2	2	*2* (3.24)*	2 (FEAS)	3 (FEAS)	2* (3.45)
bcsstk02	66	2211	–	–	–	–	*12*	–	*12 (FEAS)*	*12 (FEAS)*	*12 (FEAS)*	*12 (FEAS)*
dwt___66	66	193	–	–	2	–	2	2	2* (2.56)	2 (FEAS)	3 (FEAS)	*2* (2.43)*
west0067	67	294	–	–	*5*	–	*5*	*5*	*5 (FEAS)*	6 (FEAS)	7 (FEAS)	6 (FEAS)
dwt___72	72	147	–	–	–	–	2	–	*2* (5.75)*	2 (FEAS)	3 (FEAS)	2* (186.93)
can___73	73	225	–	–	4	–	3	3	*3* (112.13)*	4 (FEAS)	6 (FEAS)	4 (FEAS)
steam3	80	928	–	–	4	–	4	4	*4* (911.66)*	5 (FEAS)	6 (FEAS)	5 (FEAS)
ash85	85	304	–	–	3	–	2	3	*2* (18.92)*	3 (FEAS)	6 (FEAS)	3 (FEAS)
dwt___87	87	314	–	–	3	–	3	3	*3* (9.19)*	4 (FEAS)	6 (FEAS)	3* (144.45)
tols90	90	1746	–	–	–	–	*10*	–	11 (FEAS)	*10 (FEAS)*	11 (FEAS)	*10 (FEAS)*
can___96	96	432	–	–	4	–	3	3	*3* (28.30)*	4 (FEAS)	7 (FEAS)	4 (FEAS)
nos4	100	347	8	4	4	4	3	3	*3* (131.90)*	4 (FEAS)	4 (FEAS)	4 (FEAS)
olm100	100	396	–	–	–	–	2	–	*2* (555.15)*	3 (FEAS)	5 (FEAS)	3 (FEAS)
tub100	100	396	–	–	–	–	2	–	2* (164.51)	3 (FEAS)	5 (FEAS)	*2* (76.38)*
ck104	104	992	–	–	–	–	3	–	*3* (122.95)*	5 (FEAS)	6 (FEAS)	4 (FEAS)
bcsstk03	112	376	–	–	–	–	3	–	***2* (3190.90)***	3 (FEAS)	5 (FEAS)	3 (FEAS)
gent113	113	655	–	–	*5*	–	*5*	*5*	6 (FEAS)	6 (FEAS)	10 (FEAS)	6 (FEAS)

Table E.1: Detailed comparison on the $\lceil\sqrt{n}\rceil\times\lceil\sqrt{n}\rceil$ host grid for Regular and Harwell–Boeing instances (continued).

Instance	n	$\lvert E\rvert$	M3	BVNS	IG	RVNS	DRSA	RLTS	$\mathcal{F}_{\text{full}}$	Gurobi	CPLEX MIP	CPLEX CP
gre__115	115	421	–	–	–	–	*4*	–	*4 (FEAS)*	*4 (FEAS)*	8 (FEAS)	5 (FEAS)
bcspwr03	118	297	15	4	3	4	2	3	*2* (209.45)*	3 (FEAS)	6 (FEAS)	4 (FEAS)
arc130	130	1282	–	–	10	–	*9*	*9*	10 (FEAS)	10 (FEAS)	11 (FEAS)	*9 (FEAS)*
lns__131	131	536	–	–	–	–	3	–	*3* (54.92)*	4 (FEAS)	8 (FEAS)	3 (FEAS)
lnsp_131	131	536	–	–	–	–	3	–	*3* (36.66)*	4 (FEAS)	8 (FEAS)	3 (FEAS)
bcsstk04	132	1890	16	9	*8*	*8*	*8*	*8*	10 (FEAS)	11 (FEAS)	13 (FEAS)	11 (FEAS)
west0132	132	414	–	–	–	–	*5*	–	6 (FEAS)	7 (FEAS)	9 (FEAS)	7 (FEAS)
rw136	136	479	–	–	–	–	3	–	*3* (441.74)*	5 (FEAS)	8 (FEAS)	4 (FEAS)
impcol_c	137	411	–	–	–	–	*5*	–	*5 (FEAS)*	7 (FEAS)	9 (FEAS)	6 (FEAS)
bcsstk22	138	417	8	4	3	4	3	3	***2* (61.01)***	4 (FEAS)	7 (FEAS)	3 (FEAS)
can__144	144	720	16	4	4	4	*3*	4	4 (FEAS)	7 (FEAS)	9 (FEAS)	6 (FEAS)
lund_a	147	1298	–	–	*5*	–	*5*	*5*	*5 (FEAS)*	7 (FEAS)	12 (FEAS)	6 (FEAS)
lund_b	147	1294	–	–	*5*	–	*5*	*5*	*5 (FEAS)*	7 (FEAS)	12 (FEAS)	6 (FEAS)
bcsstk05	153	1288	17	7	6	*5*	6	6	7 (FEAS)	9 (FEAS)	11 (FEAS)	8 (FEAS)
west0156	156	371	–	–	–	–	*5*	–	6 (FEAS)	8 (FEAS)	10 (FEAS)	8 (FEAS)
can__161	161	769	9	6	*4*	5	*4*	*4*	*4 (FEAS)*	8 (FEAS)	10 (FEAS)	6 (FEAS)
dwt__162	162	672	–	–	–	–	3	–	*3* (322.01)*	7 (FEAS)	9 (FEAS)	4 (FEAS)
lop163	163	935	–	–	–	–	*4*	–	5 (FEAS)	6 (FEAS)	12 (FEAS)	6 (FEAS)
west0167	167	507	–	–	–	–	*5*	–	6 (FEAS)	7 (FEAS)	12 (FEAS)	8 (FEAS)
mcca	180	2659	–	–	*8*	–	*8*	*8*	*8 (FEAS)*	11 (FEAS)	13 (FEAS)	9 (FEAS)
fs_183_1	183	1069	–	–	–	–	*8*	–	9 (FEAS)	10 (FEAS)	11 (FEAS)	9 (FEAS)
gre__185	185	1005	–	–	–	–	*3*	–	4 (FEAS)	10 (FEAS)	14 (FEAS)	7 (FEAS)
can__187	187	839	–	–	–	–	3	–	*3* (1163.91)*	8 (FEAS)	14 (FEAS)	6 (FEAS)
dwt__193	193	1843	–	–	*6*	–	*6*	*6*	8 (FEAS)	15 (FEAS)	20 (FEAS)	11 (FEAS)
dwt__198	198	795	10	3	3	3	3	3	*3* (103.43)*	8 (FEAS)	11 (FEAS)	4 (FEAS)
will199	199	701	–	–	–	–	*7*	–	9 (FEAS)	13 (FEAS)	17 (FEAS)	11 (FEAS)
bwm200	200	796	–	–	–	–	2	–	2* (323.52)	6 (FEAS)	8 (FEAS)	*2* (123.14)*
rdb200	200	1120	–	–	–	–	2	–	*2* (203.08)*	6 (FEAS)	13 (FEAS)	3 (FEAS)
impcol_a	207	572	–	–	–	–	*5*	–	6 (FEAS)	9 (FEAS)	15 (FEAS)	8 (FEAS)
wm1	207	2909	–	–	10	–	*9*	10	13 (FEAS)	20 (FEAS)	23 (FEAS)	13 (FEAS)
wm2	207	2942	–	–	10	–	*9*	10	13 (FEAS)	19 (FEAS)	23 (FEAS)	13 (FEAS)
wm3	207	2948	–	–	10	–	*9*	10	13 (FEAS)	21 (FEAS)	24 (FEAS)	13 (FEAS)
dwt__209	209	976	10	6	5	5	*4*	*4*	5 (FEAS)	12 (FEAS)	18 (FEAS)	8 (FEAS)
fidap001	216	4374	–	–	–	–	*6*	–	8 (FEAS)	14 (FEAS)	22 (FEAS)	11 (FEAS)
gre_216a	216	876	–	–	–	–	*4*	–	*4 (FEAS)*	8 (FEAS)	17 (FEAS)	7 (FEAS)
ash219	219	438	–	–	–	–	*5*	–	6 (FEAS)	10 (FEAS)	17 (FEAS)	9 (FEAS)
dwt__221	221	925	10	5	4	6	*3*	4	4 (FEAS)	9 (FEAS)	20 (FEAS)	7 (FEAS)
impcol_e	225	1308	–	–	–	–	*7*	–	8 (FEAS)	12 (FEAS)	22 (FEAS)	11 (FEAS)
pde225	225	1065	–	–	–	–	1	–	1* (1701.68)	8 (FEAS)	*1* (1.98)*	8 (FEAS)
can__229	229	1003	21	5	*4*	5	*4*	*4*	*4 (FEAS)*	11 (FEAS)	18 (FEAS)	6 (FEAS)
dwt__234	234	534	21	2	3	2	3	3	*2* (743.33)*	6 (FEAS)	11 (FEAS)	3 (FEAS)
e05r0000	236	5856	–	–	–	–	*6*	–	11 (FEAS)	21 (FEAS)	26 (FEAS)	10 (FEAS)
nos1	237	627	10	4	3	4	2	3	*2* (832.54)*	4 (FEAS)	15 (FEAS)	3 (FEAS)
steam1	240	3762	–	–	*6*	–	*6*	*6*	7 (FEAS)	16 (FEAS)	23 (FEAS)	9 (FEAS)
dwt__245	245	853	11	6	4	5	*3*	4	4 (FEAS)	10 (FEAS)	17 (FEAS)	6 (FEAS)
can__256	256	1586	–	–	–	–	*7*	–	9 (FEAS)	15 (FEAS)	21 (FEAS)	13 (FEAS)
nnc261	261	1500	–	–	–	–	*4*	–	5 (FEAS)	13 (FEAS)	19 (FEAS)	7 (FEAS)
lshp_265	265	1009	11	6	3	6	*2*	3	3 (FEAS)	15 (FEAS)	20 (FEAS)	7 (FEAS)
can__268	268	1675	–	–	–	–	*6*	–	8 (FEAS)	17 (FEAS)	23 (FEAS)	10 (FEAS)
bcspwr04	274	943	12	6	4	6	*3*	4	4 (FEAS)	9 (FEAS)	20 (FEAS)	6 (FEAS)
ash292	292	1250	12	6	4	5	3	4	*3* (1197.86)*	10 (FEAS)	25 (FEAS)	6 (FEAS)
can__292	292	1416	24	7	*5*	8	*5*	*5*	6 (FEAS)	14 (FEAS)	24 (FEAS)	8 (FEAS)
dwt__307	307	1415	12	6	*4*	6	*4*	*4*	5 (FEAS)	13 (FEAS)	28 (FEAS)	9 (FEAS)
dwt__310	310	1379	12	5	4	6	*3*	4	4 (FEAS)	13 (FEAS)	29 (FEAS)	8 (FEAS)
dwt__346	346	1786	–	–	–	–	*5*	–	6 (FEAS)	23 (FEAS)	30 (FEAS)	8 (FEAS)
dwt__361	361	1657	13	7	4	6	*3*	4	11 (FEAS)	19 (FEAS)	33 (FEAS)	16 (FEAS)
plat362	362	3074	13	8	6	6	*4*	5	7 (FEAS)	26 (FEAS)	34 (FEAS)	9 (FEAS)

Table E.2: Detailed auxiliary-host results of $\mathcal{F}_{\text{full}}$, Gurobi, CPLEX MIP, and CPLEX CP under the $2\times\lceil n/2\rceil$ and $n\times n$ host grids for Regular and Harwell–Boeing instances.

Instance	n	$\lvert E\rvert$	$2\times\lceil n/2\rceil$				$n\times n$			
			$\mathcal{F}_{\text{full}}$	Gurobi	CPLEX MIP	CPLEX CP	$\mathcal{F}_{\text{full}}$	Gurobi	CPLEX MIP	CPLEX CP
Regular										
wheel5	5	8	*2* (0.06)*	2* (0.07)	2* (34.35)	2* (0.07)	*2* (0.06)*	2* (0.08)	2* (0.07)	2* (1.29)
bipartite3x3	6	9	2* (0.11)	2* (0.08)	*2* (0.05)*	2* (1.30)	*2* (0.08)*	2* (1.20)	2* (2.39)	2* (1.38)
p2xc3	6	9	*2* (0.07)*	2* (0.08)	2* (15.35)	*2* (0.07)*	*2* (0.06)*	2* (1.23)	2* (1.28)	2* (1.28)
p2xp3	6	7	*1* (0.07)*	1* (0.08)	1* (16.35)	*1* (0.07)*	*1* (0.06)*	1* (0.07)	1* (0.07)	1* (1.27)
bipartite3x4	7	12	*3* (0.06)*	3* (0.11)	3* (0.08)	3* (1.31)	*3* (1.54)*	3* (1.75)	3* (1.77)	3* (3.03)
tree2x2	7	6	*2* (0.06)*	2* (0.08)	2* (27.35)	2* (0.07)	*1* (0.06)*	1* (0.08)	*1* (0.06)*	1* (1.26)

Table E.2: Detailed auxiliary-host results under the $2 \times \lceil n/2 \rceil$ and $n \times n$ host grids (continued).

Instance	n	$\lvert E \rvert$	$2 \times \lceil n/2 \rceil$				$n \times n$			
			$\mathcal{F}_{\text{full}}$	Gurobi	CPLEX MIP	CPLEX CP	$\mathcal{F}_{\text{full}}$	Gurobi	CPLEX MIP	CPLEX CP
wheel7	7	12	*2* (0.06)*	2* (0.07)	2* (35.35)	2* (0.07)	*2* (0.05)*	2* (1.27)	2* (1.35)	2* (1.23)
bipartite4x4	8	16	*3* (0.06)*	3* (0.10)	*3* (0.06)*	3* (1.30)	*3* (1.59)*	3* (2.24)	3* (3.67)	3* (3.47)
c3xc3	9	18	*3* (0.07)*	3* (1.25)	3* (0.10)	3* (1.40)	*2* (0.08)*	2* (1.93)	2* (2.20)	2* (1.33)
p3xc3	9	15	*2* (0.06)*	2* (0.08)	2* (17.35)	2* (0.08)	*2* (0.07)*	2* (1.37)	2* (1.52)	2* (1.31)
p3xp3	9	12	*2* (0.06)*	2* (0.08)	2* (19.35)	2* (0.08)	*1* (0.07)*	1* (0.11)	1* (1.20)	1* (1.26)
bipartite5x5	10	25	4* (1.21)	4* (1.38)	*4* (1.20)*	4* (3.97)	*3* (2.02)*	3* (6.21)	3* (21.44)	3* (6.78)
cycle10	10	10	*1* (0.08)*	*1* (0.08)*	1* (4.35)	*1* (0.08)*	*1* (0.07)*	1* (1.30)	1* (1.37)	1* (1.26)
cyclePow10-10	10	45	*5* (0.06)*	5* (1.71)	5* (7.35)	5* (3.91)	*4* (98.79)*	4* (212.68)	4* (649.24)	4* (426.16)
cyclePow10-2	10	20	*2* (0.06)*	2* (0.09)	2* (8.35)	2* (0.09)	*2* (0.08)*	2* (2.28)	2* (3.34)	2* (1.30)
path10	10	9	*1* (0.06)*	1* (0.08)	1* (23.35)	1* (0.08)	*1* (0.08)*	1* (1.20)	1* (1.23)	1* (1.26)
petersen	10	15	*3* (0.06)*	3* (1.23)	3* (26.35)	3* (1.19)	*2* (0.09)*	2* (1.73)	2* (1.72)	2* (1.35)
wheel10	10	18	*3* (0.06)*	3* (0.08)	3* (31.35)	3* (0.11)	*2* (0.07)*	2* (1.73)	2* (1.61)	2* (1.29)
c3xc4	12	24	*3* (0.08)*	3* (1.27)	3* (1.22)	3* (1.59)	*2* (0.11)*	2* (3.17)	2* (2.46)	2* (1.34)
c3xk4	12	30	*4* (1.24)*	4* (1.71)	4* (1.35)	4* (4.83)	*3* (4.67)*	3* (35.96)	3* (157.89)	3* (33.75)
k3xk4	12	30	*4* (1.27)*	4* (1.55)	4* (13.35)	4* (4.70)	*3* (4.83)*	3* (36.68)	3* (237.82)	3* (53.16)
p3xk4	12	26	*2* (0.07)*	2* (1.20)	2* (18.35)	2* (0.09)	*2* (0.09)*	2* (2.40)	2* (2.02)	2* (1.34)
tree2x3	13	12	*2* (0.07)*	2* (0.09)	2* (28.35)	2* (0.10)	*2* (1.36)*	2* (8.04)	2* (18.38)	2* (4.48)
bipartite7x8	15	56	6* (179.58)	6* (0.09)	*6* (0.07)*	6* (1.25)	4 (FEAS)	*4* (928.99)*	4 (FEAS)	4 (FEAS)
cycle15	15	15	*2* (0.07)*	2* (1.47)	2* (5.35)	2* (1.21)	*2* (28.22)*	2 (FEAS)	2 (FEAS)	2* (278.46)
cyclePow15-10	15	105	*8* (0.06)*	– (TO)	8* (9.35)	– (TO)	*5 (FEAS)*	*5 (FEAS)*	*5 (FEAS)*	*5 (FEAS)*
cyclePow15-2	15	30	*3* (0.10)*	3* (2.33)	3* (10.35)	3* (1.81)	*2* (1.21)*	2* (16.24)	2* (6.39)	2* (1.40)
path15	15	14	*1* (0.07)*	1* (0.12)	1* (24.35)	1* (0.09)	*1* (0.09)*	1* (2.57)	1* (1.70)	1* (1.30)
tree3x2	15	14	*2* (0.07)*	2* (1.21)	2* (30.35)	2* (1.18)	*1* (0.09)*	1* (2.12)	1* (1.75)	1* (1.25)
wheel15	15	28	*4* (0.07)*	4* (0.11)	4* (32.35)	4* (76.10)	3* (126.04)	3 (FEAS)	*3* (42.28)*	3 (FEAS)
bipartite10x10	20	100	8* (0.17)	8* (1.21)	*8* (0.08)*	8* (1.33)	*5 (FEAS)*	*5 (FEAS)*	*5 (FEAS)*	*5 (FEAS)*
c4xc5	20	40	4* (3.52)	4* (8.86)	*4* (2.35)*	4* (1906.58)	*2* (1.52)*	2* (137.01)	2* (248.87)	2* (1.97)
c4xk5	20	60	5* (44.65)	5* (22.03)	*5* (3.35)*	– (TO)	*3* (7.64)*	3 (FEAS)	3 (FEAS)	3* (167.45)
cycle20	20	20	*1* (0.08)*	1* (1.22)	1* (6.35)	1* (0.10)	*1* (1.21)*	1* (9.22)	1* (5.09)	1* (1.35)
cyclePow20-10	20	190	*10* (0.08)*	10* (29.34)	10* (11.35)	– (TO)	*6 (FEAS)*	*6 (FEAS)*	*6 (FEAS)*	*6 (FEAS)*
cyclePow20-2	20	40	*2* (0.09)*	2* (1.52)	2* (12.35)	2* (1.24)	*2* (1.27)*	2* (27.87)	2* (17.54)	2* (1.92)
k4xk5	20	70	6 (FEAS)	6* (179.53)	*6* (14.35)*	– (TO)	*4 (FEAS)*	*4 (FEAS)*	*4 (FEAS)*	*4 (FEAS)*
p4xc5	20	35	*3* (1.29)*	3* (2.30)	3* (20.35)	3* (2.74)	*2* (1.54)*	2* (89.66)	2* (482.54)	2* (2.56)
p4xk5	20	55	*3* (0.09)*	3* (1.85)	3* (21.35)	3* (1.29)	*3* (7.14)*	3 (FEAS)	– (TO)	3* (43.01)
p4xp5	20	31	*2* (0.09)*	2* (1.69)	2* (22.35)	2* (1.22)	*1* (1.22)*	1* (17.88)	1* (4.61)	1* (1.36)
path20	20	19	*1* (0.09)*	1* (1.25)	1* (25.35)	*1* (0.09)*	*1* (1.20)*	1* (10.36)	1* (1.79)	1* (1.29)
wheel20	20	38	*6* (0.07)*	6* (1.43)	6* (33.35)	– (TO)	*3* (285.25)*	3 (FEAS)	3 (FEAS)	3 (FEAS)
tree2x4	21	20	*3* (0.07)*	3* (2.49)	3* (29.35)	3* (231.48)	*2* (1.56)*	2* (125.16)	2* (109.53)	2* (7.11)
Harwell–Boeing										
jgl009	9	50	*4* (0.10)*	4* (1.59)	4* (1.52)	4* (1.92)	*3* (1.28)*	3* (6.75)	3* (3.17)	3* (1.66)
rgg010	10	76	*5* (0.08)*	5* (1.93)	5* (1.81)	5* (12.75)	*4* (79.95)*	4* (114.31)	4* (594.88)	4* (197.21)
jgl011	11	76	*5* (1.45)*	5* (2.70)	5* (1.71)	5* (9.25)	*4* (127.24)*	4* (349.30)	4 (FEAS)	4* (738.78)
can___24	24	92	*3* (1.31)*	3* (4.63)	3* (3.08)	3* (4.26)	*2* (1.35)*	2* (164.95)	2 (FEAS)	2* (1.45)
lap_25	25	97	*4* (1.21)*	4* (52.32)	4* (25.70)	4* (145.02)	*2* (1.39)*	2* (373.56)	2 (FEAS)	2* (1.91)
fidap005	27	279	*5* (2.25)*	5* (40.80)	5* (323.79)	5* (239.33)	*4* (1482.38)*	4 (FEAS)	4 (FEAS)	4 (FEAS)
pores_1	30	180	*4* (1.89)*	4* (70.24)	4 (FEAS)	4* (446.92)	*3* (13.54)*	3 (FEAS)	4 (FEAS)	3* (131.12)
ibm32	32	126	*6 (FEAS)*	*6 (FEAS)*	7 (FEAS)	*6 (FEAS)*	*3* (462.15)*	4 (FEAS)	4 (FEAS)	3 (FEAS)
bcspwr01	39	85	*3* (3.75)*	3* (178.98)	3 (FEAS)	3 (FEAS)	*2* (2.33)*	2 (FEAS)	2 (FEAS)	2* (7.11)
fidapm05	42	520	*6* (8.77)*	6* (661.87)	7 (FEAS)	6 (FEAS)	*4 (FEAS)*	5 (FEAS)	5 (FEAS)	*4 (FEAS)*
bcsstk01	48	224	*9 (FEAS)*	– (TO)	*9 (FEAS)*	*9 (FEAS)*	*4 (FEAS)*	5 (FEAS)	6 (FEAS)	*4 (FEAS)*
bcspwr02	49	108	*4* (1037.80)*	4 (FEAS)	5 (FEAS)	4 (FEAS)	*2* (4.13)*	2 (FEAS)	4 (FEAS)	2* (9.58)
curtis54	54	291	5 (FEAS)	*5* (1030.38)*	8 (FEAS)	6 (FEAS)	*3* (41.30)*	4 (FEAS)	6 (FEAS)	3 (FEAS)
will57	57	281	*4* (10.97)*	– (TO)	6 (FEAS)	4 (FEAS)	*3* (126.32)*	4 (FEAS)	5 (FEAS)	3 (FEAS)
dwt___59	59	163	*3* (4.16)*	3* (339.85)	4 (FEAS)	3 (FEAS)	*2* (8.05)*	3 (FEAS)	5 (FEAS)	2* (36.98)
impcol_b	59	312	– (TO)	– (TO)	– (TO)	– (TO)	*4 (FEAS)*	6 (FEAS)	8 (FEAS)	5 (FEAS)
can___61	61	309	*7* (37.33)*	7 (FEAS)	9 (FEAS)	7 (FEAS)	*4 (FEAS)*	*4 (FEAS)*	7 (FEAS)	*4 (FEAS)*
bfw62a	62	450	*7 (FEAS)*	*7 (FEAS)*	8 (FEAS)	*7 (FEAS)*	*3* (61.27)*	5 (FEAS)	7 (FEAS)	4 (FEAS)
bfw62b	62	342	*5* (434.41)*	5 (FEAS)	7 (FEAS)	5 (FEAS)	*3* (290.19)*	5 (FEAS)	6 (FEAS)	3* (1153.74)
can___62	62	140	*3* (36.54)*	3* (1070.51)	6 (FEAS)	3 (FEAS)	*2* (11.60)*	4 (FEAS)	5 (FEAS)	2* (36.10)
bcsstk02	66	2211	*33* (0.06)*	33 (FEAS)	33 (FEAS)	33 (FEAS)	*11 (FEAS)*	– (TO)	– (TO)	*11 (FEAS)*
dwt___66	66	193	*2* (17.44)*	2* (622.91)	4 (FEAS)	2* (147.26)	*2* (10.88)*	4 (FEAS)	7 (FEAS)	2* (19.53)
west0067	67	294	– (TO)	– (TO)	– (TO)	– (TO)	*5 (FEAS)*	7 (FEAS)	10 (FEAS)	*5 (FEAS)*
dwt___72	72	147	*3* (108.82)*	3 (FEAS)	7 (FEAS)	3 (FEAS)	1* (24.21)	4 (FEAS)	8 (FEAS)	*1* (6.20)*
can___73	73	225	*9 (FEAS)*	*9 (FEAS)*	11 (FEAS)	*9 (FEAS)*	*3 (FEAS)*	5 (FEAS)	9 (FEAS)	*3 (FEAS)*
steam3	80	928	*3* (30.95)*	3* (67.49)	5 (FEAS)	3 (FEAS)	*2* (19.36)*	7 (FEAS)	16 (FEAS)	2* (221.05)
ash85	85	304	*5 (FEAS)*	6 (FEAS)	11 (FEAS)	*5 (FEAS)*	*2* (37.20)*	6 (FEAS)	12 (FEAS)	2* (205.00)
dwt___87	87	314	*5 (FEAS)*	7 (FEAS)	9 (FEAS)	6 (FEAS)	*3* (757.44)*	6 (FEAS)	42 (FEAS)	3 (FEAS)
tols90	90	1746	– (TO)	– (TO)	– (TO)	– (TO)	*9 (FEAS)*	– (TO)	– (TO)	*9 (FEAS)*
can___96	96	432	10 (FEAS)	*8 (FEAS)*	– (TO)	9 (FEAS)	*3 (FEAS)*	– (TO)	14 (FEAS)	*3 (FEAS)*
nos4	100	347	*5 (FEAS)*	*5 (FEAS)*	14 (FEAS)	6 (FEAS)	*2* (82.79)*	10 (FEAS)	11 (FEAS)	2 (FEAS)
olm100	100	396	*2* (31.35)*	3 (FEAS)	6 (FEAS)	4 (FEAS)	*2* (54.22)*	5 (FEAS)	3 (FEAS)	2* (312.55)
tub100	100	396	*1* (26.47)*	1* (948.40)	2 (FEAS)	4 (FEAS)	1* (10.36)	6 (FEAS)	2 (FEAS)	*1* (3.02)*
ck104	104	992	*3* (809.80)*	9 (FEAS)	7 (FEAS)	7 (FEAS)	*3* (911.67)*	– (TO)	– (TO)	3 (FEAS)
bcsstk03	112	376	*2* (693.32)*	3 (FEAS)	11 (FEAS)	5 (FEAS)	*2* (96.65)*	– (TO)	49 (FEAS)	2* (244.31)

Table E.2: Detailed auxiliary-host results under the $2 \times \lceil n/2 \rceil$ and $n \times n$ host grids (continued).

Instance	n	$\lvert E \rvert$	$2 \times \lceil n/2 \rceil$				$n \times n$			
			$\mathcal{F}_{\text{full}}$	Gurobi	CPLEX MIP	CPLEX CP	$\mathcal{F}_{\text{full}}$	Gurobi	CPLEX MIP	CPLEX CP
gent113	113	655	– (TO)	– (TO)	– (TO)	*14 (FEAS)*	*6 (FEAS)*	– (TO)	– (TO)	*6 (FEAS)*
gre__115	115	421	15 (FEAS)	*14 (FEAS)*	– (TO)	*14 (FEAS)*	*3 (FEAS)*	– (TO)	69 (FEAS)	4 (FEAS)
bcspwr03	118	297	*6 (FEAS)*	7 (FEAS)	11 (FEAS)	7 (FEAS)	*2* (160.18)*	– (TO)	– (TO)	2 (FEAS)
arc130	130	1282	– (TO)	*32 (FEAS)*	– (TO)	*32 (FEAS)*	9 (FEAS)	– (TO)	– (TO)	*8 (FEAS)*
lns__131	131	536	*11 (FEAS)*	*11 (FEAS)*	– (TO)	12 (FEAS)	*3* (2227.35)*	– (TO)	– (TO)	3 (FEAS)
lnsp_131	131	536	*11 (FEAS)*	14 (FEAS)	– (TO)	12 (FEAS)	*3* (2280.70)*	– (TO)	– (TO)	3 (FEAS)
bcsstk04	132	1890	– (TO)	– (TO)	– (TO)	– (TO)	*8 (FEAS)*	– (TO)	– (TO)	9 (FEAS)
west0132	132	414	– (TO)	– (TO)	– (TO)	– (TO)	*6 (FEAS)*	– (TO)	– (TO)	7 (FEAS)
rw136	136	479	*9 (FEAS)*	*9 (FEAS)*	– (TO)	12 (FEAS)	*2* (349.05)*	– (TO)	– (TO)	2 (FEAS)
impcol_c	137	411	– (TO)	*16 (FEAS)*	– (TO)	– (TO)	*5 (FEAS)*	– (TO)	– (TO)	*5 (FEAS)*
bcsstk22	138	417	*6 (FEAS)*	*6 (FEAS)*	– (TO)	8 (FEAS)	*2* (379.35)*	– (TO)	– (TO)	3 (FEAS)
can__144	144	720	*10 (FEAS)*	11 (FEAS)	– (TO)	13 (FEAS)	*3* (1394.24)*	– (TO)	– (TO)	4 (FEAS)
lund_a	147	1298	*12 (FEAS)*	– (TO)	– (TO)	16 (FEAS)	*4 (FEAS)*	– (TO)	– (TO)	6 (FEAS)
lund_b	147	1294	16 (FEAS)	– (TO)	– (TO)	*15 (FEAS)*	*4 (FEAS)*	– (TO)	– (TO)	6 (FEAS)
bcsstk05	153	1288	16 (FEAS)	– (TO)	– (TO)	*12 (FEAS)*	–	–	–	–
west0156	156	371	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
can__161	161	769	*15 (FEAS)*	– (TO)	– (TO)	*15 (FEAS)*	–	–	–	–
dwt__162	162	672	*12 (FEAS)*	17 (FEAS)	– (TO)	*12 (FEAS)*	–	–	–	–
lop163	163	935	*8 (FEAS)*	9 (FEAS)	– (TO)	10 (FEAS)	–	–	–	–
west0167	167	507	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
mcca	180	2659	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
fs_183_1	183	1069	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
gre__185	185	1005	*15 (FEAS)*	– (TO)	– (TO)	19 (FEAS)	–	–	–	–
can__187	187	839	14 (FEAS)	– (TO)	– (TO)	*12 (FEAS)*	–	–	–	–
dwt__193	193	1843	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
dwt__198	198	795	*13 (FEAS)*	16 (FEAS)	– (TO)	*13 (FEAS)*	–	–	–	–
will199	199	701	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
bwm200	200	796	13 (FEAS)	– (TO)	*1* (392.73)*	13 (FEAS)	–	–	–	–
rdb200	200	1120	*12 (FEAS)*	– (TO)	– (TO)	16 (FEAS)	–	–	–	–
impcol_a	207	572	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
wm1	207	2909	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
wm2	207	2942	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
wm3	207	2948	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
dwt__209	209	976	20 (FEAS)	– (TO)	– (TO)	*19 (FEAS)*	–	–	–	–
fidap001	216	4374	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
gre_216a	216	876	*18 (FEAS)*	– (TO)	– (TO)	19 (FEAS)	–	–	–	–
ash219	219	438	*19 (FEAS)*	– (TO)	– (TO)	– (TO)	–	–	–	–
dwt__221	221	925	*14 (FEAS)*	– (TO)	– (TO)	19 (FEAS)	–	–	–	–
impcol_e	225	1308	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
pde225	225	1065	*14 (FEAS)*	– (TO)	– (TO)	*14 (FEAS)*	–	–	–	–
can__229	229	1003	21 (FEAS)	– (TO)	– (TO)	*19 (FEAS)*	–	–	–	–
dwt__234	234	534	*13 (FEAS)*	– (TO)	– (TO)	14 (FEAS)	–	–	–	–
e05r0000	236	5856	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
nos1	237	627	19 (FEAS)	16 (FEAS)	– (TO)	*13 (FEAS)*	–	–	–	–
steam1	240	3762	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
dwt__245	245	853	*18 (FEAS)*	– (TO)	– (TO)	*18 (FEAS)*	–	–	–	–
can__256	256	1586	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
nnc261	261	1500	22 (FEAS)	– (TO)	– (TO)	*19 (FEAS)*	–	–	–	–
lshp_265	265	1009	*19 (FEAS)*	– (TO)	– (TO)	21 (FEAS)	–	–	–	–
can__268	268	1675	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
bcspwr04	274	943	*19 (FEAS)*	– (TO)	– (TO)	– (TO)	–	–	–	–
ash292	292	1250	*14 (FEAS)*	– (TO)	– (TO)	– (TO)	–	–	–	–
can__292	292	1416	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
dwt__307	307	1415	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
dwt__310	310	1379	*24 (FEAS)*	– (TO)	– (TO)	– (TO)	–	–	–	–
dwt__346	346	1786	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
dwt__361	361	1657	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–
plat362	362	3074	– (TO)	– (TO)	– (TO)	– (TO)	–	–	–	–

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used ChatGPT by OpenAI to assist with language revision. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., Pollitt, F., 2024. CaDiCaL 2.0, in: Computer Aided Verification (CAV 2024), Springer. pp. 133–152. doi:10.1007/978-3-031-65627-9_7.

[2] Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), 2021. Handbook of Satisfiability. volume 336 of *Frontiers in Artificial Intelligence and Applications*. 2 ed., IOS Press. doi:10.3233/FAIA336.

[3] Cavero, S., Pardo, E.G., Duarte, A., 2023. Efficient iterated greedy for the two-dimensional bandwidth minimization problem. European Journal of Operational Research 306, 1126–1139. doi:10.1016/j.ejor.2022.09.004.

[4] Cavero, S., Pardo, E.G., Rodriguez-Tello, E., Monfroy, E., Vasconcellos-Gaete, C., 2026. Optimizing the two-dimensional bandwidth problem under the maximum norm using exact and heuristic approaches. European Journal of Operational Research doi:10.1016/j.ejor.2026.06.012.

[5] Chung, F.R.K., 1988. Labelings of graphs, in: Beineke, L.W., Wilson, R.J. (Eds.), Selected Topics in Graph Theory. Academic Press. volume 3, pp. 151–168.

[6] Faber, D., Jabrayilov, A., Mutzel, P., 2024. SAT encoding of partial ordering models for graph coloring problems, in: 27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024), Schloss Dagstuhl – Leibniz-Zentrum für Informatik. pp. 12:1–12:20. doi:10.4230/LIPIcs.SAT.2024.12.

[7] Fazekas, K., Sinnl, M., Biere, A., Parragh, S., 2020. Duplex encoding of staircase at-most-one constraints for the antibandwidth problem, in: Integration of Constraint Programming, Artificial Intelligence, and Operations Research, Springer. pp. 186–204. doi:10.1007/978-3-030-58942-4_13.

[8] Fichte, J.K., Le Berre, D., Hecher, M., Szeider, S., 2023. The silent (R)evolution of SAT. Communications of the ACM 66, 64–72. doi:10.1145/3560469.

[9] Gurobi Optimization, LLC, 2025. Gurobi Optimizer Reference Manual. https://docs.gurobi.com/projects/optimizer/en/12.0/. Version 12.0.3, accessed 15 August 2026.

[10] IBM Corporation, 2022. IBM ILOG CPLEX Optimization Studio. https://www.ibm.com/docs/en/icos/22.1.1. Version 22.1.1, accessed 15 August 2026.

[11] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437. doi:10.1007/978-3-319-94144-8_26.

[12] Ignatiev, A., Tan, Z.L., Karamanos, C., 2024. Towards universally accessible SAT technology, in: 27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024), Schloss Dagstuhl – Leibniz-Zentrum für Informatik. pp. 16:1–16:11. doi:10.4230/LIPIcs.SAT.2024.16.

[13] Khandelwal, A., Srivastava, K., Saran, G., 2023a. Grid bandwidth minimization problem: Simulated annealing approach. Mapana Journal of Sciences 22, 101–128. doi:10.12723/mjs.sp1.9.

[14] Khandelwal, A., Srivastava, K., Saran, G., 2023b. A reduced variable neighbourhood search algorithm for grid bandwidth minimization problem, in: 2023 7th International Conference on Intelligent Computing and Control Systems (ICICCS), pp. 795–800. doi:10.1109/ICICCS56967.2023.10142562.

[15] Lin, L., Lin, Y., 2010. Two models of two-dimensional bandwidth problems. Information Processing Letters 110, 469–473. doi:10.1016/j.ipl.2010.04.013.

[16] Lin, L., Lin, Y., 2011. Square-root rule of two-dimensional bandwidth problem. RAIRO – Theoretical Informatics and Applications 45, 399–411. doi:10.1051/ita/2011120.

[17] Lin, Y., 1996. On density lower bounds of two dimensional bandwidth. Journal of Mathematical Research and Exposition 16, 343–349.

[18] Lin, Y., Hao, J., Li, X., 2000. Two-dimensional bandwidth problem under distance of L_∞-norm. OR Transactions 4, 8–12. doi:10.15960/j.cnki.issn.1007-6093.2000.03.002.

[19] National Institute of Standards and Technology, 2002. Matrix Market: Sparse matrix collection. https://math.nist.gov/MatrixMarket/matrices.html. Accessed 15 August 2026.

[20] Niewiadomski, A., Switalski, P., Sidoruk, T., Penczek, W., 2019. Applying modern SAT-solvers to solving hard problems. Fundamenta Informaticae 165, 321–344. doi:10.3233/FI-2019-1788.

[21] Prestwich, S.D., 2009. CNF encodings, in: Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), Handbook of Satisfiability. IOS Press. volume 185 of *Frontiers in Artificial Intelligence and Applications*, pp. 75–97. doi:10.3233/978-1-58603-929-5-75.

[22] Rodríguez-García, M.Á., Duarte, A., Sánchez-Oro, J., 2018. GRASP with path relinking for 2D-bandwidth minimization problem, in: Proceedings of the International Conference on Learning and Optimization Algorithms: Theory and Applications, Association for Computing Machinery, New York, NY, USA. pp. 52:1–52:5. doi:10.1145/3230905.3230953.

[23] Rodríguez-García, M.Á., Sánchez-Oro, J., Rodriguez-Tello, E., Monfroy, É., Duarte, A., 2021. Two-dimensional bandwidth minimization problem: Exact and heuristic approaches. Knowledge-Based Systems 214, 106651. doi:10.1016/j.knosys.2020.106651.

[24] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. doi:10.1007/11564751_73.

[25] Torres-Jimenez, J., Blanco-Rocha, R., Lara-Alvarez, C., Izquierdo-Marquez, I., 2025. Two dimensional bandwidth minimization problem using a novel evaluation function. Expert Systems with Applications 290, 127986. doi:10.1016/j.eswa.2025.127986.

[26] Zhou, Q., Gao, M., Hao, J.K., 2026a. Reinforcement learning driven heuristic for two-dimensional bandwidth minimization. Applied Soft Computing 187, 114327. doi:10.1016/j.asoc.2025.114327.

[27] Zhou, Q., Gao, M., Hao, J.K., 2026b. Reinforcement learning driven heuristic for two-dimensional bandwidth minimization. https://github.com/neteasefans/two-dimensional-bandwidth-minimization-problem. Software and benchmark-data repository, accessed 15 August 2026.

Research Data



Click here to download Research data
https://github.com/pqmlmaoxd/two-dimensional-bandwidth-minimization-